



Intermediate Machine Learning: Assignment 3

Deadline

Assignment 3 is due Thursday, October 30 by 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a pdf file on Gradescope, and as a notebook (.ipynb) on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this [converter notebook](#).

Topics

- Variational autoencoders (VAEs)
- Undirected graphs
- The graphical lasso

Problem 1: Face time (35 points)

In this problem, we will implement a "shoestring" version of [this amazing fake face generator](#), using a variational autoencoder (VAE). Building a generator like the one featured in the article can take a tremendous amount of computational resources, time, and parameter tuning. In this problem we will build a basic version to illustrate the main concepts, and help you to become more familiar with VAEs. Here is an outline of the process that we'll step you through:

Problem outline:

- Load data
- Create face groups based on attributes
- Construct the VAE
- Define the loss function and train the VAE (Problem 1.1)
- Encode and reconstruct faces (Problem 1.2)
- Visualize the latent space (Problem 1.3)
- Morph between faces (Problem 1.4)
- Shift attributes of faces (Problem 1.5)
- Generate new faces (Problem 1.6)
- Analyze the effect of the scaling factor in the loss function (Problem 1.7, optional)

In the next cell we load the packages that we'll need. If you don't have one or more of these, you can install them with `!pip install <package_name>` in the cell, or outside the notebook with `conda install -c conda-forge <package_name>`

```
In [1]: import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import os
import glob
import pandas as pd
import random
import numpy as np
import imageio
from tqdm import tqdm
from PIL import Image
```

```
from sklearn.model_selection import train_test_split
from scipy.stats import norm
import tensorflow
# tensorflow.compat.v1.disable_eager_execution()
```

Loading the data

[Labeled Faces in the Wild](#) (LFW) is a database of face photographs. The images are placed in the folder lfw-deepfunneled. lfw_attributes.txt is a document including a set of attributes associated for each image, such as 'Male', 'Smile', 'Bold', etc. All the features are numerical and large positive values indicate that the keywords well describe the photo; large negative values indicate that the keywords don't fit the photo.

For this problem, we will keep only the middle parts of the photos to avoid complex backgrounds.

Download the data from the cloud at these URLs:

<https://sds365.s3.amazonaws.com/lfw/lfw-deepfunneled.zip>
https://sds365.s3.amazonaws.com/lfw/lfw_attributes.txt

Once you have the data, unzip it, and place it in a directory that we will call "YOUR_PATH" below.

Run all the cells in this section to load the data.

Note: Please write down the entire path instead of using something like '~/Desktop/datasets/' to avoid unnecessary compiling errors. Also, if you choose to use Colab to do your homework. We need to download the data into the same directory as your code. You may also need

"from google.colab import drive"

"drive.mount('/content/drive')"

to enable using paths in Google Drive before starting your code below. But personally, I would suggest using jupyter notebook to run the code locally instead of using Google Colab since the latter may take longer time.

```
In [2]: # Change these path names to correspond with your directory
DATASET_PATH = "/Users/gitano/Downloads/lfw-deepfunneled/"
ATTRIBUTES_PATH = "/Users/gitano/Downloads/lfw_attributes.txt"
```

```
In [3]: # Make sure the above path is correct before running this cell
dataset = []
for path in glob.iglob(os.path.join(DATASET_PATH, "**", "*.jpg")):
    person = path.split("/")[-2]
    dataset.append({"person":person, "path": path})

dataset = pd.DataFrame(dataset)
dataset = dataset.groupby("person").filter(lambda x: len(x) < 100 )
dataset.head(10)
```

Out [3]:

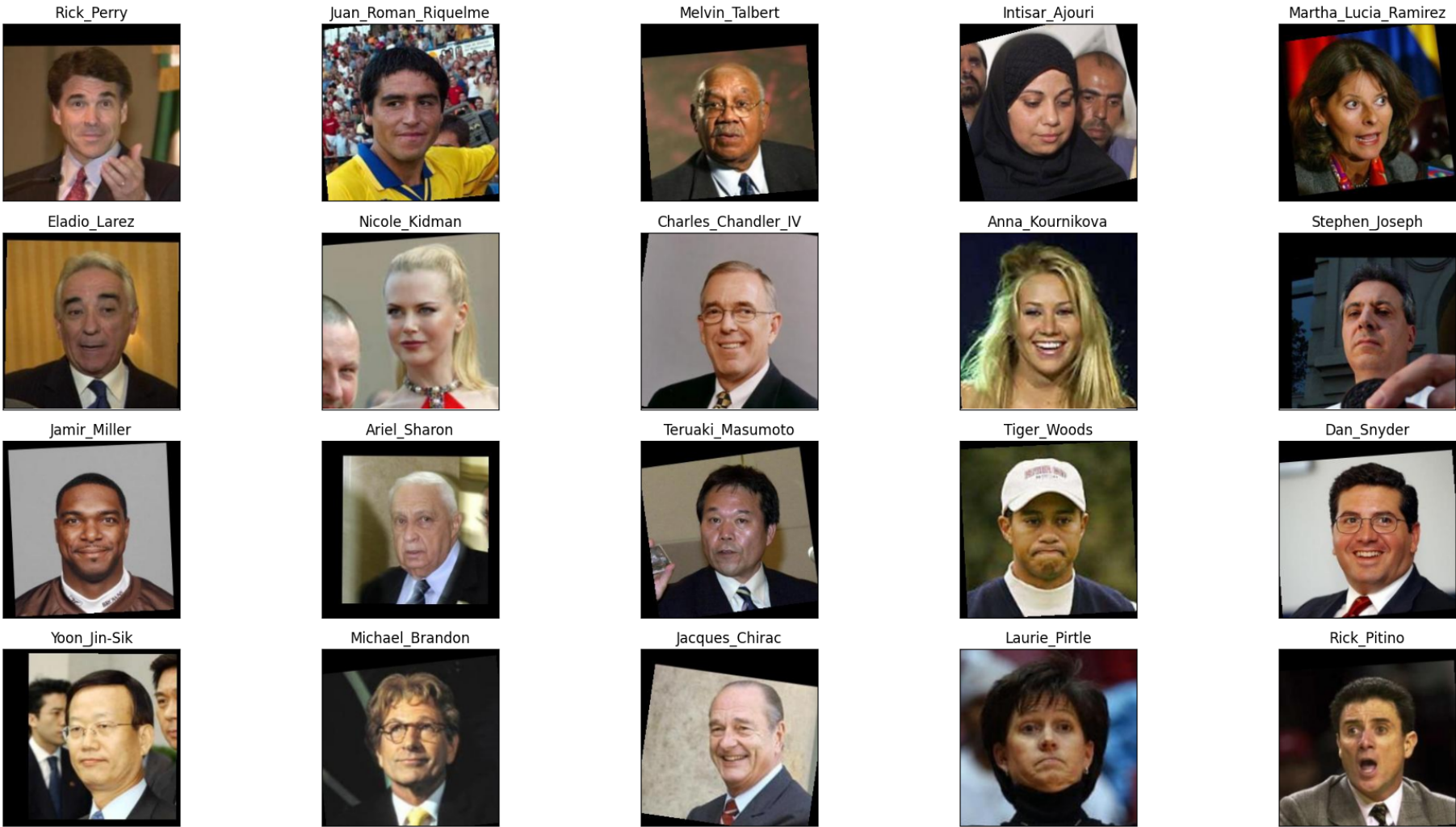
	person	path
0	German_Khan	/Users/gitano/Downloads/lfw-deepfunneled/Germa...
1	Stefano_Gabbana	/Users/gitano/Downloads/lfw-deepfunneled/Stefa...
2	Dragan_Covic	/Users/gitano/Downloads/lfw-deepfunneled/Draga...
3	Jeff_Hornacek	/Users/gitano/Downloads/lfw-deepfunneled/Jeff_...
4	Sureyya_Ayhan	/Users/gitano/Downloads/lfw-deepfunneled/Surey...
5	Deb_Santos	/Users/gitano/Downloads/lfw-deepfunneled/Deb_S...
6	Bob_Newhart	/Users/gitano/Downloads/lfw-deepfunneled/Bob_N...
7	Wang_Hailan	/Users/gitano/Downloads/lfw-deepfunneled/Wang_...
8	Paul_McNulty	/Users/gitano/Downloads/lfw-deepfunneled/Paul_...
9	Paul_McNulty	/Users/gitano/Downloads/lfw-deepfunneled/Paul_...

The following cell will display some sample images

```
In [4]: sampled_id = []

plt.figure(figsize=(20,10))
for i in range(20):
    idx = random.randint(0, len(dataset))
    img = plt.imread(dataset.path.iloc[idx])
```

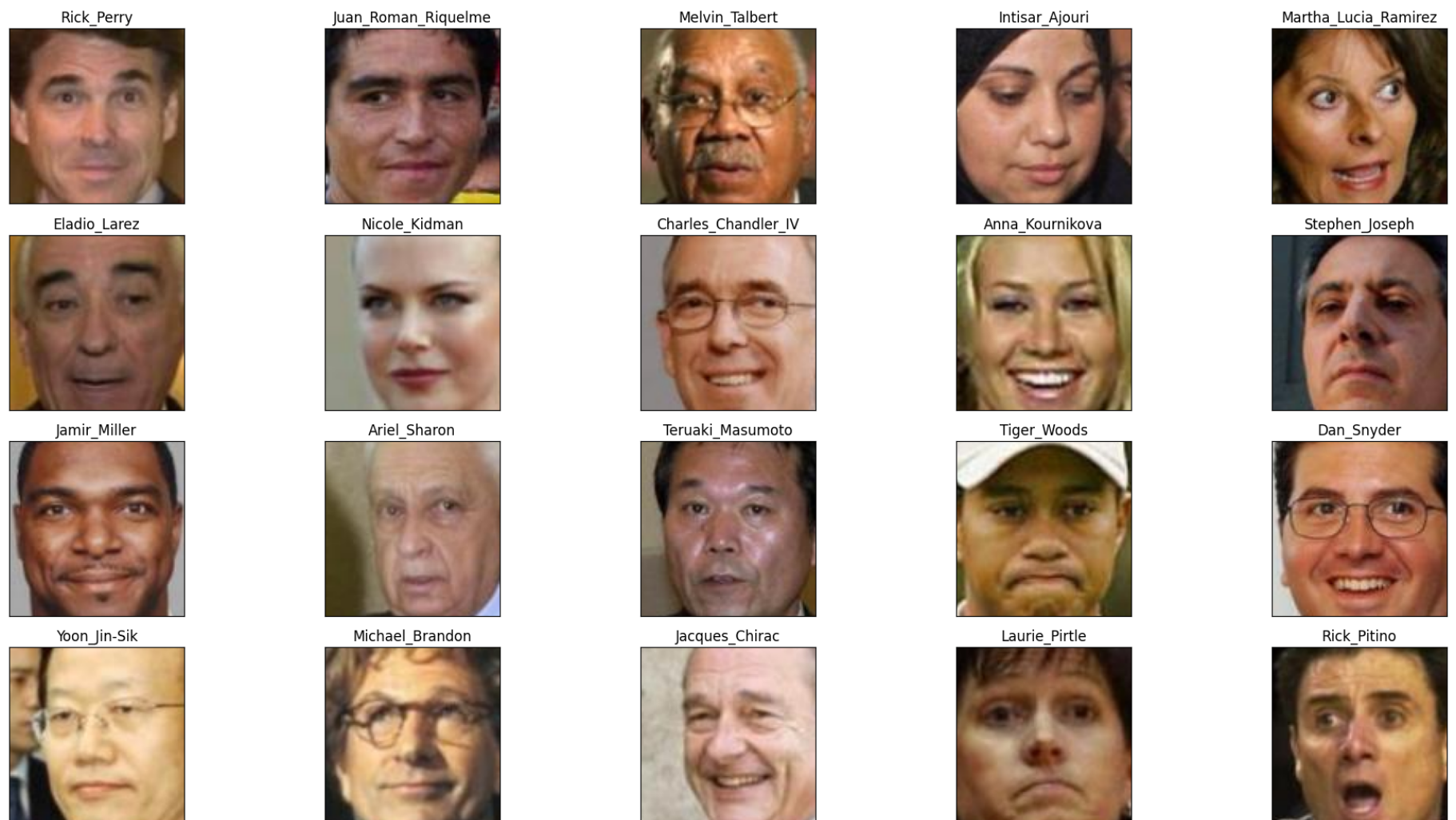
```
plt.subplot(4, 5, i+1)
plt.imshow(img)
plt.title(dataset.person.iloc[idx])
plt.xticks([])
plt.yticks([])
sampled_id.append(idx)
plt.tight_layout()
plt.show()
```



The following cell shows the images with some of the background removed.

```
In [5]: dx=70
dy=70

plt.figure(figsize=(20,10))
for i in range(20):
    idx = sampled_id[i]
    img = plt.imread(dataset.path.iloc[idx])
    plt.subplot(4, 5, i+1)
    plt.imshow(img[dg:-dg,dx:-dx])
    plt.title(dataset.person.iloc[idx])
    plt.xticks([])
    plt.yticks([])
plt.tight_layout()
plt.show()
```

The following function crops the images to 45x45 pixels, which is what we will use in this problem.

```
In [4]: def fetch_dataset(dx=70, dy=70, dimx=45, dimy=45):

    df_attrs = pd.read_csv(ATTRIBUTES_PATH, sep='\t', skiprows=1,)
    df_attrs = pd.DataFrame(df_attrs.iloc[:, :-1].values, columns = df_attrs.columns[1:])

    photo_ids = []
    for dirpath, dirnames, filenames in os.walk(DATASET_PATH):
        for fname in filenames:
            if fname.endswith(".jpg"):
                fpath = os.path.join(dirpath, fname)
                photo_id = fname[:-4].replace('_', ' ').split()
                person_id = ' '.join(photo_id[:-1])
                photo_number = int(photo_id[-1])
                photo_ids.append({'person': person_id, 'imagenum': photo_number, 'photo_path': fpath})

    photo_ids = pd.DataFrame(photo_ids)
    df = pd.merge(df_attrs, photo_ids, on=('person', 'imagenum'))

    assert len(df) == len(df_attrs), "lost some data when merging dataframes"

    all_photos = df['photo_path'].apply(imageio.imread)\
        .apply(lambda img: img[dg:-dy, dx:-dx])\
        .apply(lambda img: np.array(Image.fromarray(img).resize([dimx, dimy])))

    all_photos = np.stack(all_photos.values).astype('uint8')
    all_attrs = df.drop(["photo_path", "person", "imagenum"], axis=1)

    return all_photos, all_attrs
```

The variable `data` has all the face images and the variable `attrs` has all the attributes. The 8-bit RGB values are converted to values between 0 and 1 for modeling and plotting purposes.

```
In [5]: data, attrs = fetch_dataset()
data = np.array(data / 255, dtype='float32')
```

```
In [58]: # print the attributes names
print(attrs.columns.tolist())
```

```
['Male', 'Asian', 'White', 'Black', 'Baby', 'Child', 'Youth', 'Middle Aged', 'Senior', 'Black Hair', 'Blond Hair', 'Brown Hair', 'Bald', 'No Eyewear', 'Eyeglasses', 'Sunglasses', 'Mustache', 'Smiling', 'Frowning', 'Chubby', 'Blurry', 'Harsh Lighting', 'Flash', 'Soft Lighting', 'Outdoor', 'Curly Hair', 'Wavy Hair', 'Straight Hair', 'Receding Hairline', 'Bangs', 'Sideburns', 'Fully Visible Forehead', 'Partially Visible Forehead', 'Obstructed Forehead', 'Bushy Eyebrows', 'Arched Eyebrows', 'Narrow Eyes', 'Eyes Open', 'Big Nose', 'Pointy Nose', 'Big Lips', 'Mouth Closed', 'Mouth Slightly Open', 'Mouth Wide Open', 'Teeth Not Visible', 'No Beard', 'Goatee', 'Round Jaw', 'Double Chin', 'Wearing Hat', 'Oval Face', 'Square Face', 'Round Face', 'Color Photo', 'Posed Photo', 'Attractive Man', 'Attractive Woman', 'Indian', 'Gray Hair', 'Bags Under Eyes', 'Heavy Makeup', 'Rosy Cheeks', 'Shiny Skin', 'Pale Skin', '5 o' Clock Shadow', 'Strong Nose-Mouth Lines', 'Wearing Lipstick', 'Flushed Face', 'High Cheekbones', 'Brown Eyes', 'Wearing Earrings', 'Wearing Necktie', 'Wearing Necklace']
```


Create Face Groups

We can now create groups of faces, by selecting the faces having the highest or lowest scores for each of the attributes. Run all the cells in this section to create and plot some face groups.

```
In [6]: def plot_gallery(images, h, w, n_row=3, n_col=6, with_title=False, titles=[]):
plt.figure(figsize=(1.75 * n_col, 2 * n_row))
plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)
for i in range(n_row * n_col):
    plt.subplot(n_row, n_col, i + 1)
    try:
        plt.imshow(images[i].reshape((h, w, 3)), cmap=plt.cm.gray, vmin=-1, vmax=1, interpolation='nearest')
        if with_title:
            plt.title(titles[i])
        plt.xticks(())
        plt.yticks(())
    except:
        pass
```

```
In [7]: IMAGE_H = data.shape[1]
IMAGE_W = data.shape[2]
N_CHANNELS = 3
```

```
In [8]: smile_ids = attrs['Smiling'].sort_values(ascending=False).head(36).index.values
smile_data = data[smile_ids]

no_smile_ids = attrs['Smiling'].sort_values(ascending=True).head(36).index.values
no_smile_data = data[no_smile_ids]

eyeglasses_ids = attrs['Eyeglasses'].sort_values(ascending=False).head(36).index.values
eyeglasses_data = data[eyeglasses_ids]

sunglasses_ids = attrs['Sunglasses'].sort_values(ascending=False).head(36).index.values
sunglasses_data = data[sunglasses_ids]

mustache_ids = attrs['Mustache'].sort_values(ascending=False).head(36).index.values
mustache_data = data[mustache_ids]

male_ids = attrs['Male'].sort_values(ascending=False).head(36).index.values
male_data = data[male_ids]

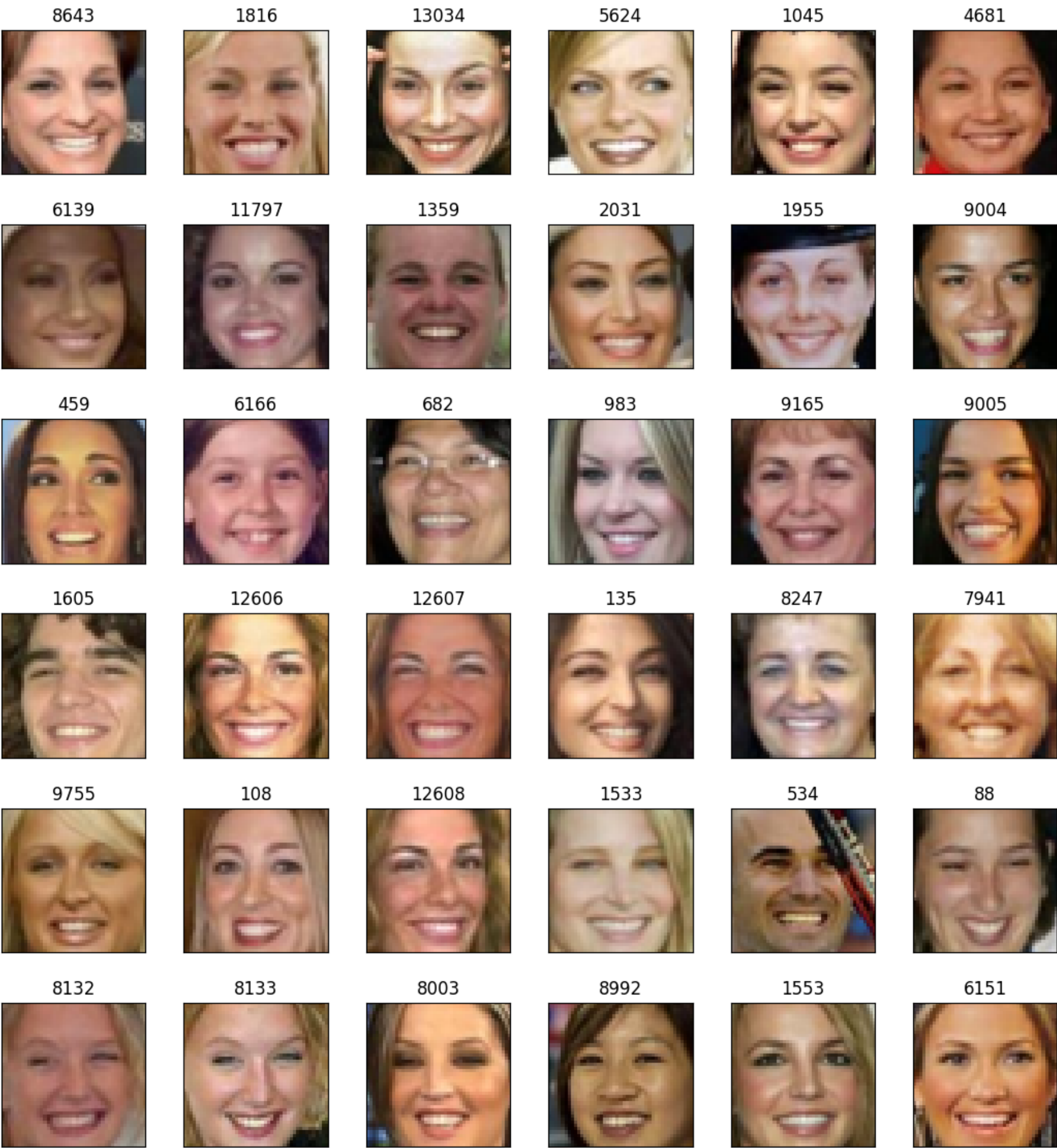
female_ids = attrs['Male'].sort_values(ascending=True).head(36).index.values
female_data = data[female_ids]

eyeclosed_ids = attrs['Eyes Open'].sort_values(ascending=True).head(36).index.values
eyeclosed_data = data[eyeclosed_ids]

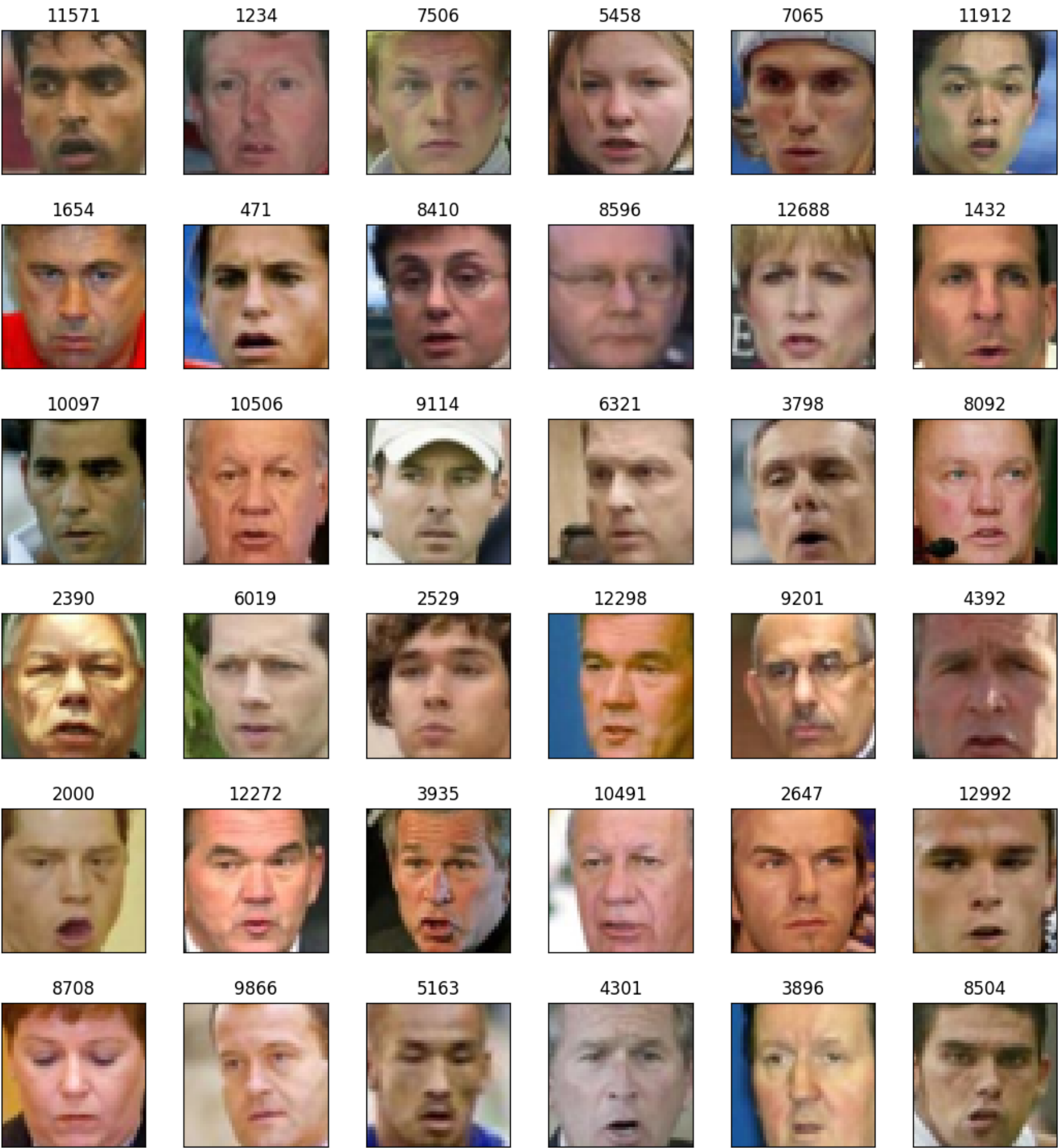
mouthopen_ids = attrs['Mouth Wide Open'].sort_values(ascending=False).head(36).index.values
mouthopen_data = data[mouthopen_ids]

makeup_ids = attrs['Heavy Makeup'].sort_values(ascending=False).head(36).index.values
makeup_data = data[makeup_ids]
```

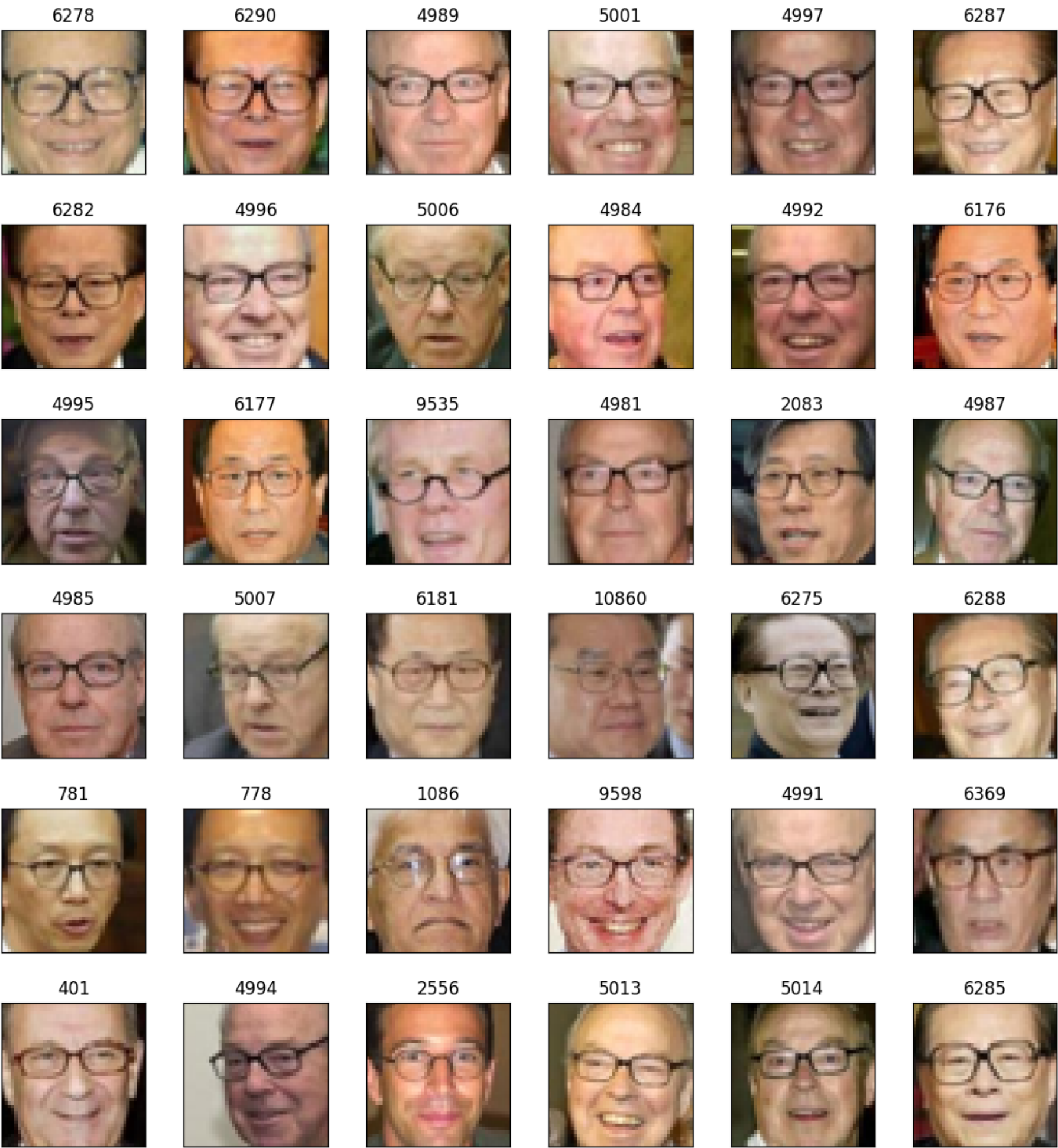
```
In [11]: plot_gallery(smile_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=smile_ids)
```



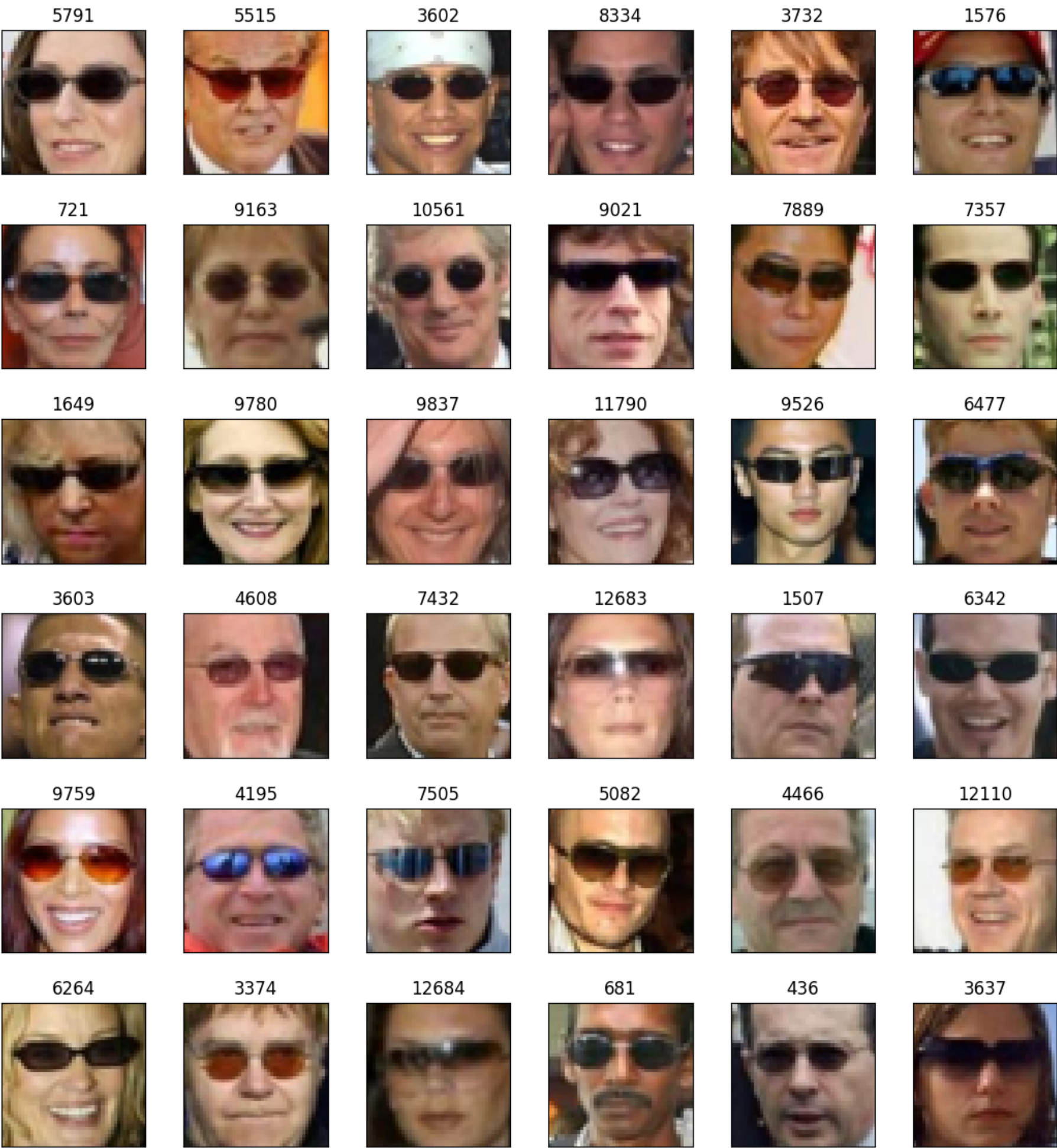
```
In [12]: plot_gallery(no_smile_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=no_smile_ids)
```



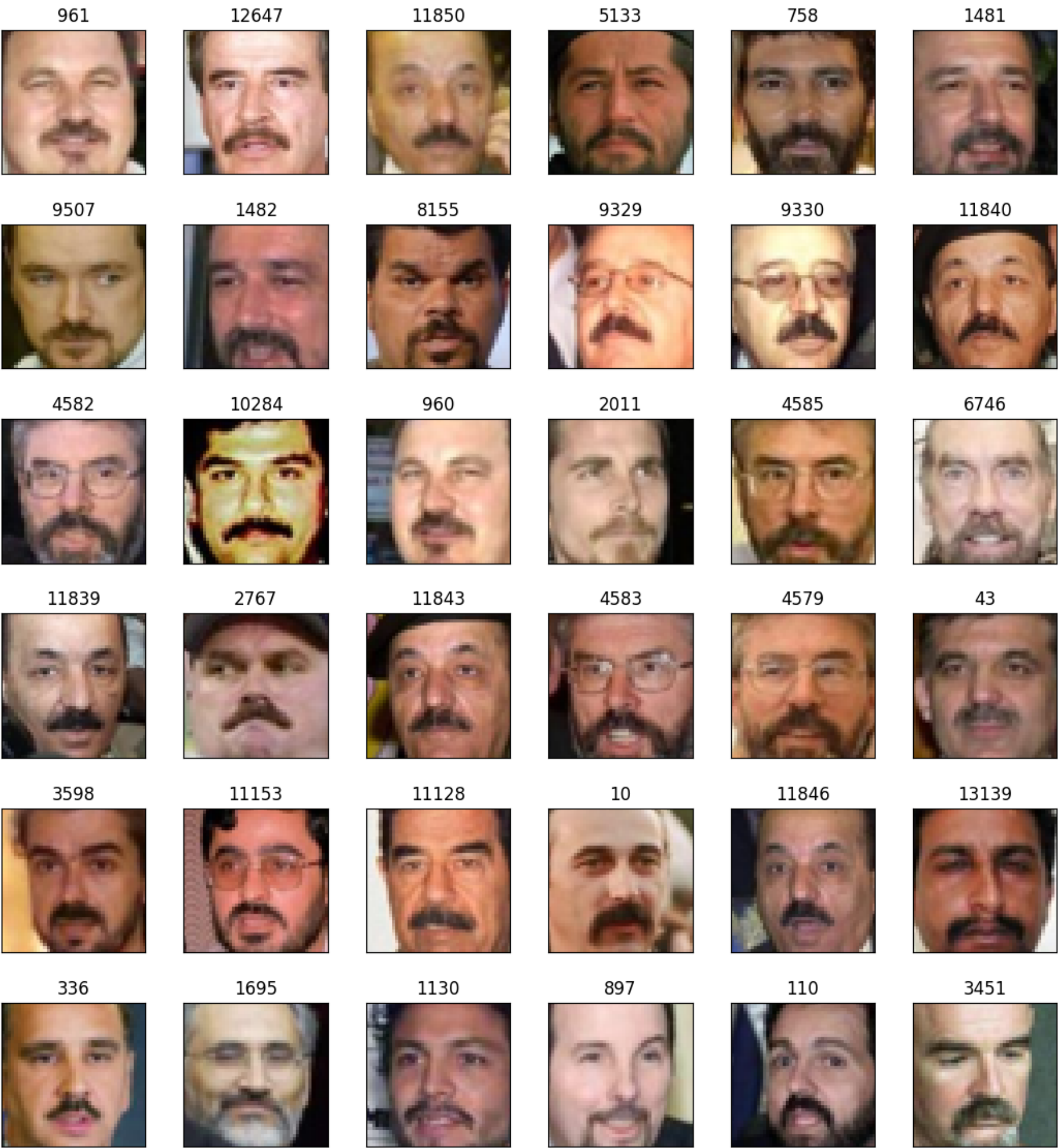
```
In [13]: plot_gallery(eyeglasses_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=eyeglasses_ids)
```

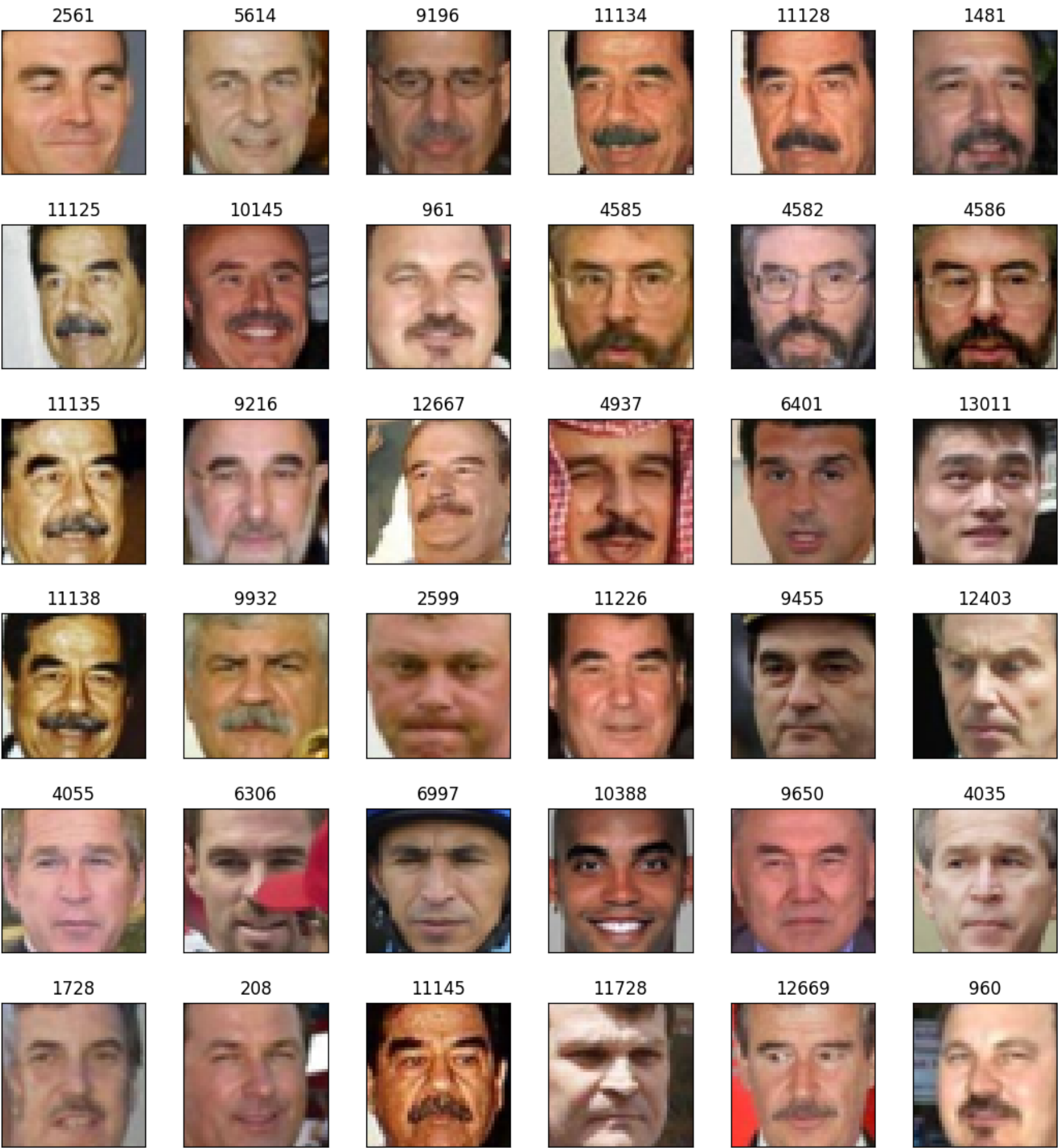
```
In [14]: plot_gallery(sunglasses_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=sunglasses_ids)
```



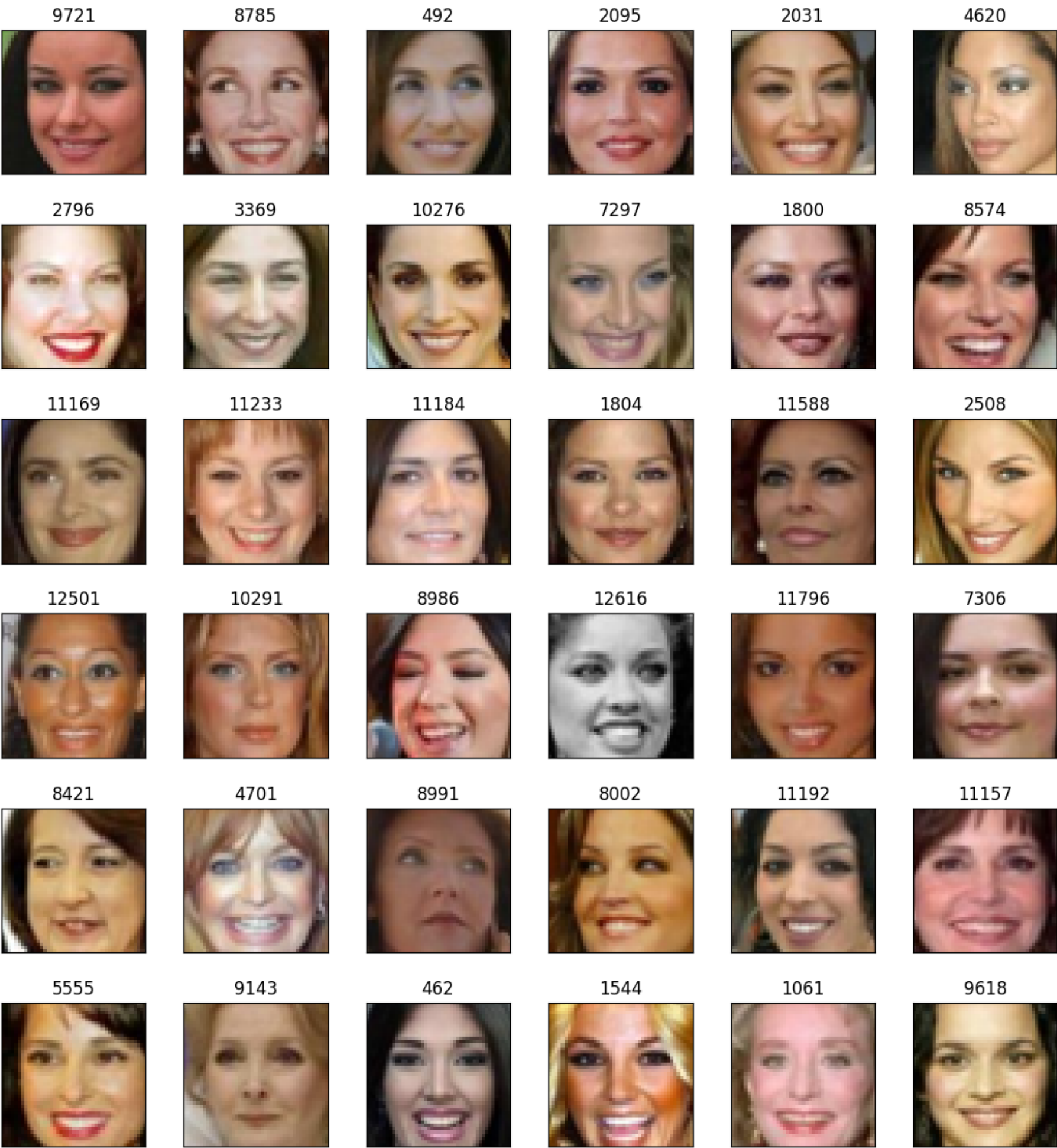
```
In [15]: plot_gallery(mustache_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=mustache_ids)
```



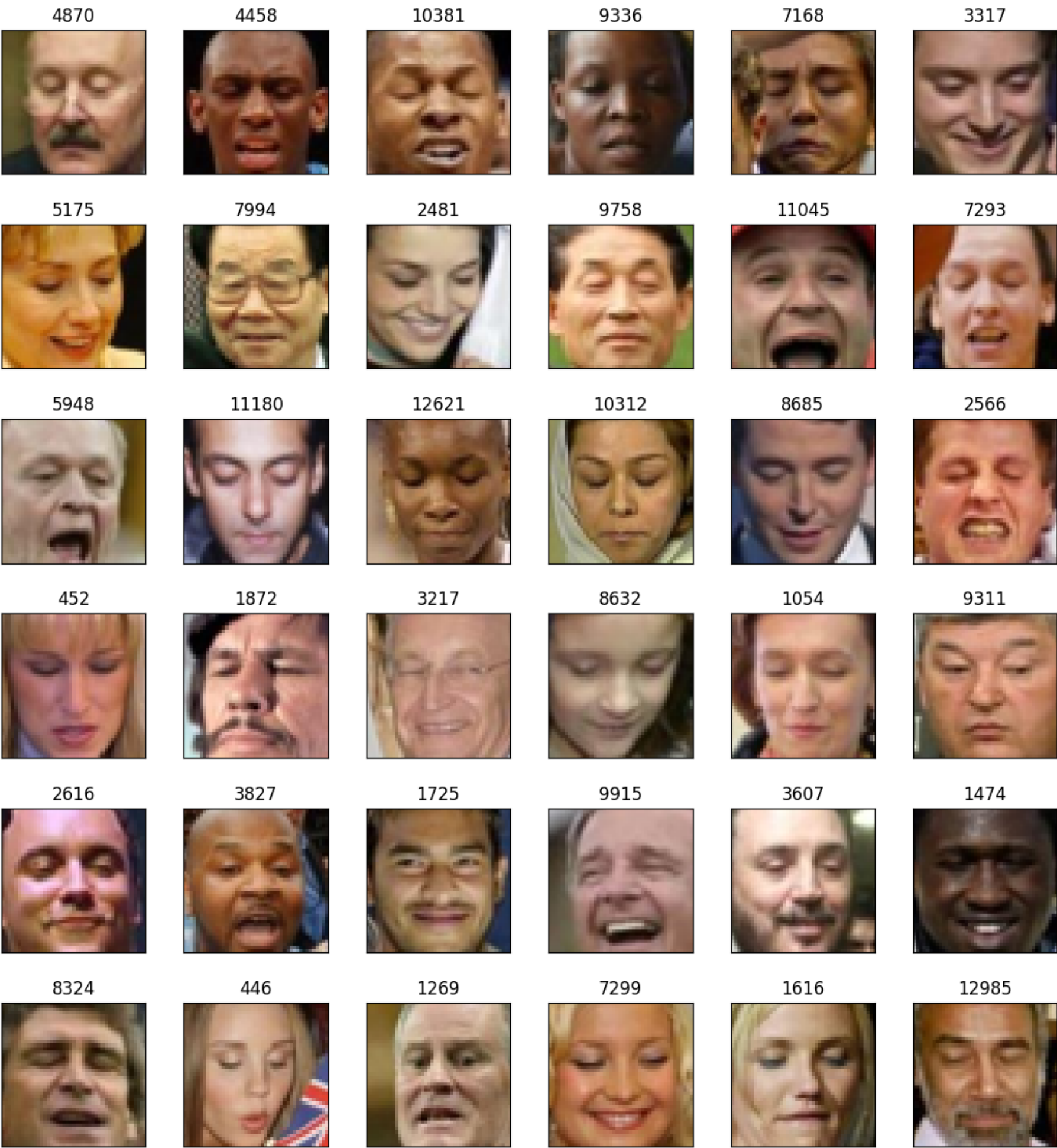
```
In [16]: plot_gallery(male_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=male_ids)
```

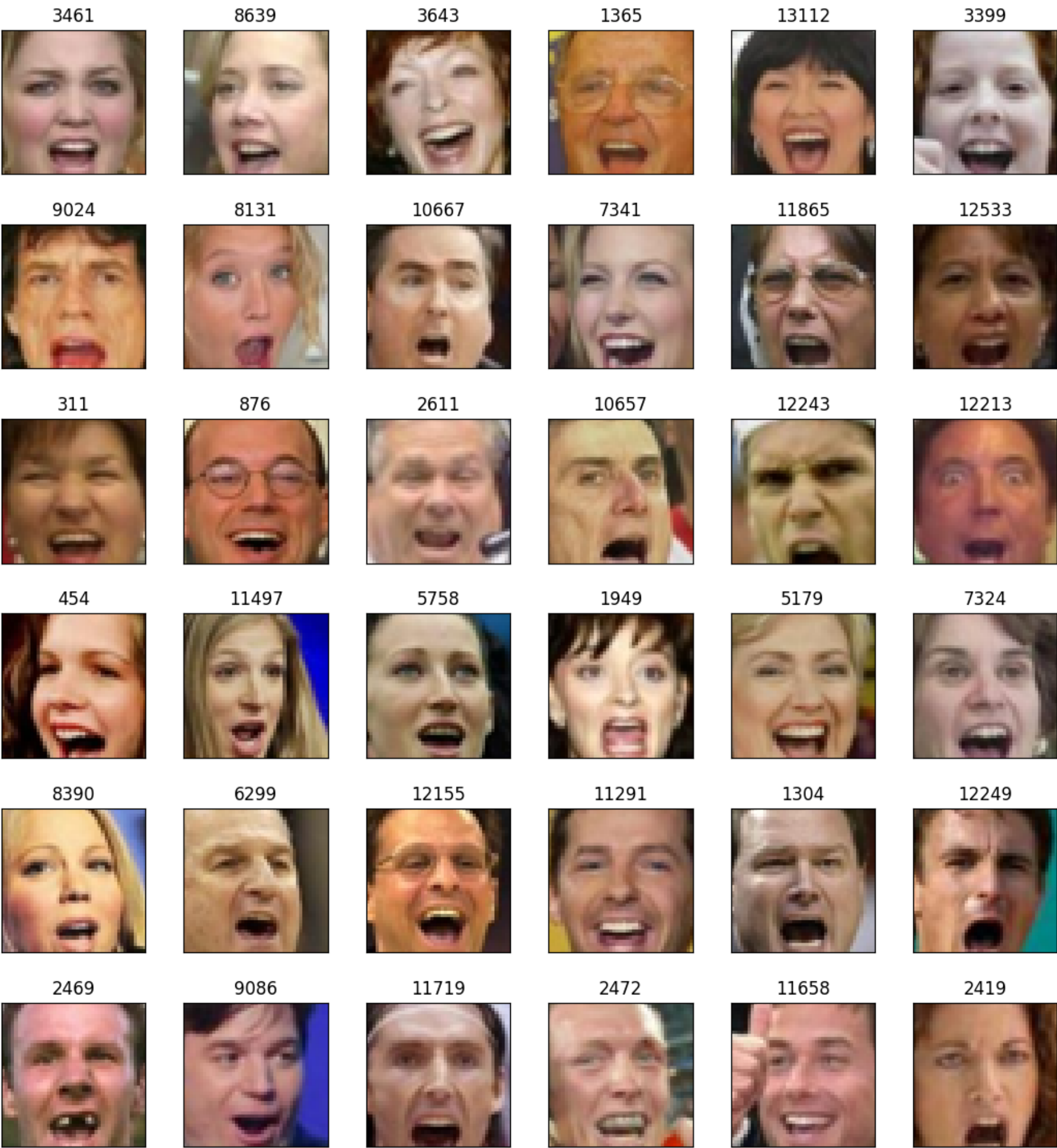
```
In [17]: plot_gallery(female_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=female_ids)
```

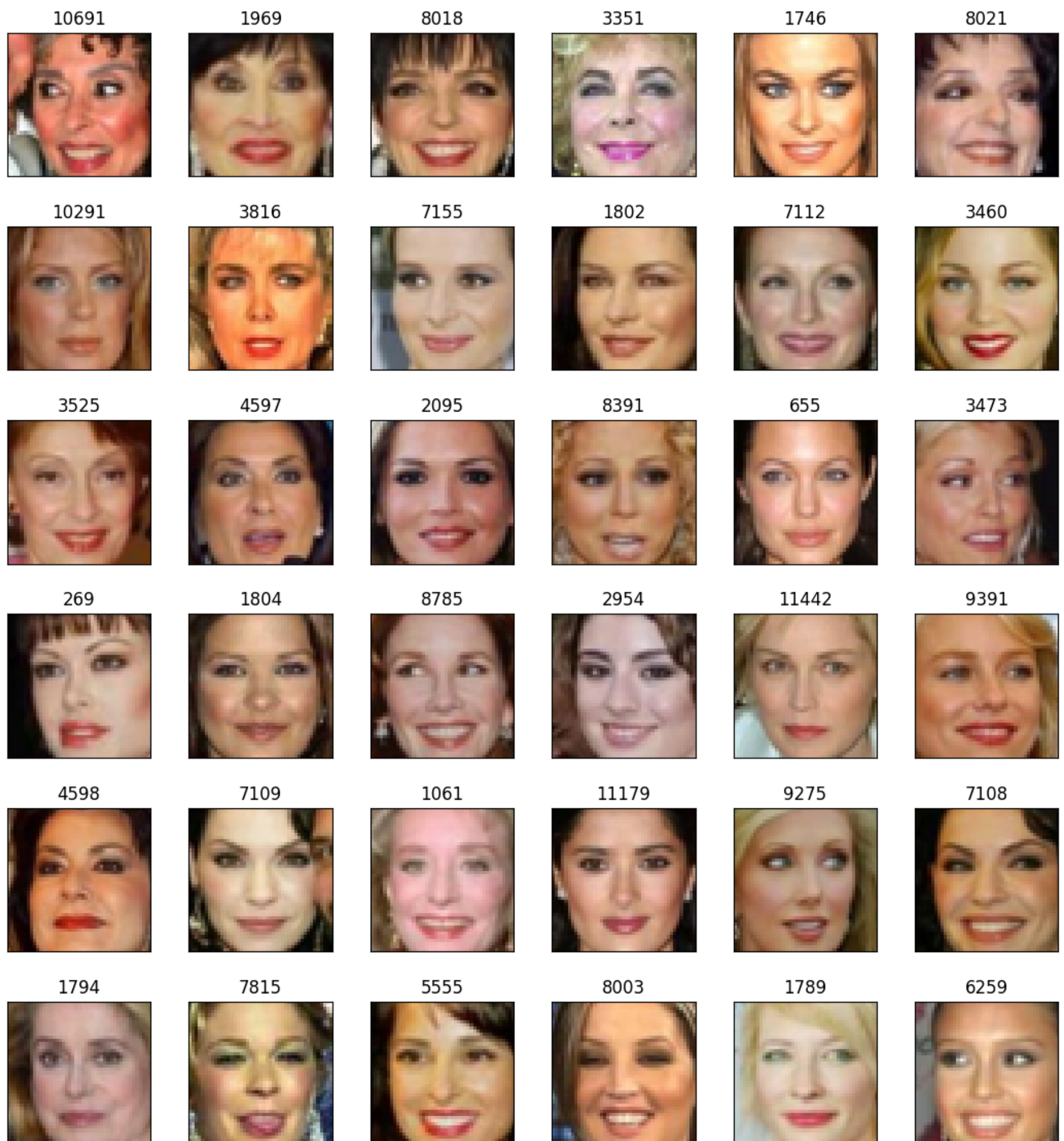
```
In [18]: plot_gallery(eyeclosed_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=eyeclosed_ids)
```



```
In [19]: plot_gallery(mouthopen_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=mouthopen_ids)
```



```
In [20]: plot_gallery(makeup_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True, titles=makeup_ids)
```

Constructing the encoder

Recall that the encoder part of the VAE architecture maps a data point to a variational mean and (log) variance. The mean is a point in the latent space.

```
In [9]: LATENT_SPACE_SIZE = 100
```

The "reparameterization trick" draws samples from the variational distribution that are parameterized by the variational mean and variance, so that the parameters of the encoder network can be trained.

```
In [10]: # @tensorflow.keras.utils.register_keras_serializable()
def sample_latent_features(distribution):
    distribution_mean, distribution_variance = distribution
    batch_size = tensorflow.shape(distribution_variance)[0]
    random = tensorflow.keras.backend.random_normal(shape=(batch_size, tensorflow.shape(distribution_variance)[1]))
    return distribution_mean + tensorflow.exp(0.5 * distribution_variance) * random
```

```
In [11]: input_data = tensorflow.keras.layers.Input(shape=(45, 45, 3))

encoder = tensorflow.keras.layers.Conv2D(64, (5,5), activation='relu')(input_data)
encoder = tensorflow.keras.layers.MaxPooling2D((2,2))(encoder)

encoder = tensorflow.keras.layers.Conv2D(64, (3,3), activation='relu')(encoder)
encoder = tensorflow.keras.layers.MaxPooling2D((2,2))(encoder)
```

```
encoder = tensorflow.keras.layers.Conv2D(32, (3,3), activation='relu')(encoder)
encoder = tensorflow.keras.layers.MaxPooling2D((2,2))(encoder)

encoder = tensorflow.keras.layers.Flatten()(encoder)

distribution_mean = tensorflow.keras.layers.Dense(LATENT_SPACE_SIZE, name='variational_mean')(encoder)
distribution_variance = tensorflow.keras.layers.Dense(LATENT_SPACE_SIZE, name='variational_log_variance')(encoder)
latent_encoding = tensorflow.keras.layers.Lambda(sample_latent_features)([distribution_mean, distribution_variance])

encoder_model = tensorflow.keras.Model(input_data, latent_encoding)
encoder_model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 45, 45, 3)	0	–
conv2d (Conv2D)	(None, 41, 41, 64)	4,864	input_layer[0][0]
max_pooling2d (MaxPooling2D)	(None, 20, 20, 64)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 18, 18, 64)	36,928	max_pooling2d[0]...
max_pooling2d_1 (MaxPooling2D)	(None, 9, 9, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 7, 7, 32)	18,464	max_pooling2d_1[...
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 32)	0	conv2d_2[0][0]
flatten (Flatten)	(None, 288)	0	max_pooling2d_2[...
variational_mean (Dense)	(None, 100)	28,900	flatten[0][0]
variational_log_va... (Dense)	(None, 100)	28,900	flatten[0][0]
lambda (Lambda)	(None, 100)	0	variational_mean... variational_log_...

Total params: 118,056 (461.16 KB)
Trainable params: 118,056 (461.16 KB)
Non-trainable params: 0 (0.00 B)

Construct the decoder

The decoder network in the VAE architecture maps a latent vector to an image. This is done with a series of transposed convolutional layers, since it must map from low to high dimensions.

```
In [12]: decoder_input = tensorflow.keras.layers.Input(shape=(LATENT_SPACE_SIZE,))
decoder = tensorflow.keras.layers.Reshape((1, 1, 100))(decoder_input)
decoder = tensorflow.keras.layers.Conv2DTranspose(64, (3,3), activation='relu')(decoder)
decoder = tensorflow.keras.layers.UpSampling2D((2,2))(decoder)

decoder = tensorflow.keras.layers.Conv2DTranspose(32, (3,3), activation='relu')(decoder)
decoder = tensorflow.keras.layers.UpSampling2D((2,2))(decoder)

decoder = tensorflow.keras.layers.Conv2DTranspose(16, (5,5), activation='relu')(decoder)
decoder = tensorflow.keras.layers.UpSampling2D((2,2))(decoder)

decoder_output = tensorflow.keras.layers.Conv2DTranspose(3, (6,6), activation='relu')(decoder)

decoder_model = tensorflow.keras.Model(decoder_input, decoder_output)
decoder_model.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 100)	0
reshape (Reshape)	(None, 1, 1, 100)	0
conv2d_transpose (Conv2DTranspose)	(None, 3, 3, 64)	57,664
up_sampling2d (UpSampling2D)	(None, 6, 6, 64)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 8, 8, 32)	18,464
up_sampling2d_1 (UpSampling2D)	(None, 16, 16, 32)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 20, 20, 16)	12,816
up_sampling2d_2 (UpSampling2D)	(None, 40, 40, 16)	0
conv2d_transpose_3 (Conv2DTranspose)	(None, 45, 45, 3)	1,731

Total params: 90,675 (354.20 KB)
Trainable params: 90,675 (354.20 KB)
Non-trainable params: 0 (0.00 B)

```
In [40]: encoded = encoder_model(input_data)
         decoded = decoder_model(encoded)
         autoencoder = tensorflow.keras.models.Model(input_data, decoded)

In [41]: # @tensorflow.keras.utils.register_keras_serializable()
         def get_loss(distribution_mean, distribution_variance, factor, batch_size):

             def get_reconstruction_loss(y_true, y_pred, factor, batch_size):
                 reconstruction_loss = tensorflow.math.squared_difference(y_true, y_pred)
                 reconstruction_loss_batch = tensorflow.reduce_sum(reconstruction_loss)/batch_size
                 return 0.5*reconstruction_loss_batch*factor

             def get_kl_loss(distribution_mean, distribution_variance, batch_size):
                 kl_loss = LATENT_SPACE_SIZE + distribution_variance - tensorflow.square(distribution_mean) - tensorflow.exp(distribution_variance)
                 kl_loss_batch = tensorflow.reduce_sum(kl_loss)/batch_size
                 return kl_loss_batch*(-0.5)

             def total_loss(y_true, y_pred):
                 reconstruction_loss_batch = get_reconstruction_loss(y_true, y_pred, factor, batch_size)
                 kl_loss_batch = get_kl_loss(distribution_mean, distribution_variance, batch_size)
                 return reconstruction_loss_batch + kl_loss_batch

             return total_loss

In [42]: from keras import ops, layers, Model
         from keras.losses import mean_squared_error as mse

         # --- KL layer ---
         # @tensorflow.keras.utils.register_keras_serializable()
         class KLLoss(layers.Layer):
             def __init__(self, beta=1.0, **kwargs):
                 super().__init__(**kwargs)
                 self.beta = beta

             def call(self, inputs):
                 mu, logvar = inputs
                 # KL per sample: -0.5 * sum(1 + logvar - mu^2 - exp(logvar))
                 kl_per_sample = -0.5 * ops.sum(1.0 + logvar - ops.square(mu) - ops.exp(logvar), axis=-1)
                 self.add_loss(self.beta * ops.mean(kl_per_sample)) # scalar KL
                 return inputs # pass-through

         # 1) Create the KL node
         kl_node = KLLoss(name="kl_loss")([distribution_mean, distribution_variance])

         # 2) Tie the KL node into the forward graph but keep output = decoded
         decoded = layers.Lambda(lambda xs: xs[0], name="with_kl")([decoded, kl_node])

         # 3) (Re)build the model so the KL loss is included
         autoencoder = tensorflow.keras.models.Model(input_data, decoded, name="autoencoder_with_kl")

         # 4) reconstruction loss
         def reconstruction_loss(y_true, y_pred, sum_over_channels=False):
```



```
# per-pixel MSE (averaged over channels)
per_pixel = mse(y_true, y_pred) # (B, H, W)
H = ops.shape(per_pixel)[1]
W = ops.shape(per_pixel)[2]
factor = ops.cast(H * W, per_pixel.dtype)
if sum_over_channels:
    C = ops.shape(y_true)[-1] # undo the channel mean
    factor = factor * ops.cast(C, per_pixel.dtype)
return ops.mean(per_pixel) * factor # mean over batch, sum over chosen dims
```

1.1 Deriving the loss function (5 points)

Derive the loss function defined in the cell above from the probability model perspective. You can ignore the scalar `factor` in your derivation. Show your work using either LaTeX or a picture of your written solution.

Hint: Think about how the total loss is related to the ELBO.

1.1

Set the size of latent space is D . the size of data dimension is n .

In the encoder, we suppose that $q(z|x) = \mathcal{N}(\mu(x), \sigma^2 I_D)$

In the decoder, we suppose that $z \sim \mathcal{N}(0, I_D)$. $x|z \sim \mathcal{N}(G(z), I_n)$

The distribution variance in the code is $\log(\sigma^2)$.

From the code. $Loss_R = \frac{\sum_i (\hat{x}_i - x_i)^2}{\text{batch-size}} \times \frac{1}{2}$ $Loss_{KL} = \frac{\sum_{i=1}^D (1 + \log \sigma_i^2 - \mu_i^2 - \exp(\log \sigma_i^2))}{\text{batch-size}} \times (-\frac{1}{2})$

$$\textcircled{1} -\log p(x|z) = -\log \left((2\pi)^{-n/2} \|I_n\|^{-1/2} \exp\left(-\frac{1}{2} (x-G(z))^T I_n^{-1} (x-G(z))\right) \right) = -\log \left((2\pi)^{-n/2} \exp\left(-\frac{1}{2} \|x-G(z)\|^2\right) \right) \\ = \frac{1}{2} \|x-G(z)\|^2 + \text{const}$$

From the LLN: $-E_q(\log p(x|z)) \approx \frac{1}{N} \sum_{i=1}^N -\log p(x|z) = \frac{1}{N} \cdot N \cdot \left(\frac{1}{2} \|x-G(z)\|^2 + \text{const} \right) = \frac{1}{2} \|x-G(z)\|^2 + \text{const}$

$$\textcircled{2} -E_q(\log p(z)) = -E_q\left(\log \left((2\pi)^{-D/2} \|I_D\|^{-1/2} \exp\left(-\frac{1}{2} z^T I_D^{-1} z\right) \right)\right) = \frac{1}{2} E_q \|z\|^2 + \text{const} \\ = \frac{1}{2} E_q \left(\sum_{j=1}^D z_j^2 \right) + \text{const} = \frac{1}{2} \sum_{j=1}^D E_q(z_j^2) + \text{const}$$

we have $z_j \sim \mathcal{N}(\mu_j, \sigma^2)$, $\text{Var}(z_j) = E(z_j^2) - E^2(z_j)$, so $E_q(z_j^2) = \text{Var}(z_j) + E^2(z_j) = \sigma^2 + \mu_j^2$

$$\text{So } -E_q(\log p(z)) = \frac{1}{2} \sum_{j=1}^D (\sigma^2 + \mu_j^2) + \text{const} = \frac{1}{2} (D \cdot \sigma^2 + \| \mu \|^2) + \text{const} = \frac{D}{2} \sigma^2 + \frac{1}{2} \| \mu \|^2 + \text{const}$$

$$\textcircled{3} H(q) = -E_q(\log q(z)) = -E_q(\log q(z|x)) = -E_q\left(\log \left((2\pi)^{-D/2} \|\sigma^2 I_D\|^{-1/2} \exp\left(-\frac{1}{2} (z-\mu(x))^T (\sigma^2 I_D)^{-1} (z-\mu(x))\right) \right)\right) \\ = \frac{1}{2\sigma^2} (\|z-\mu(x)\|^2) + \frac{D}{2} \log(2\pi\sigma^2)$$

We have $z|x \sim \mathcal{N}(\mu(x), \sigma^2 I_D) \Rightarrow z - \mu(x) | x \sim \mathcal{N}(0, \sigma^2 I_D) \Rightarrow z_j - \mu_j(x) | x \sim \mathcal{N}(0, \sigma^2)$

$$\text{Var}\left(\frac{z_j - \mu_j(x)}{\sigma}\right) = E\left(\left(\frac{z_j - \mu_j(x)}{\sigma}\right)^2\right) - E^2\left(\frac{z_j - \mu_j(x)}{\sigma}\right) = E\left(\frac{z_j - \mu_j(x)}{\sigma}\right)^2 - 0 = E\left(\frac{z_j - \mu_j(x)}{\sigma}\right)^2$$

$$H(q) = \frac{1}{2\sigma^2} E_q\left(\sum_{j=1}^D (z_j - \mu_j(x))^2\right) + \frac{D}{2} \log(2\pi\sigma^2) = \frac{1}{2\sigma^2} \sum_{j=1}^D E_q(z_j - \mu_j(x))^2 + \frac{D}{2} \log(2\pi\sigma^2) \\ = \frac{1}{2\sigma^2} D \sigma^2 + \frac{D}{2} \log(2\pi\sigma^2) = \frac{D}{2} \log \sigma^2 + \frac{D}{2}$$

$$\Rightarrow ELBO = H(q) + E_q(\log p(x, z)) = H(q) + \sum_z q(z) \log p(x, z)$$

$$= H(q) + \sum_z q(z) \log p(x|z) + \sum_z q(z) \log p(z)$$

$$= H(q) + E_q(\log p(x|z)) + E_q(\log p(z))$$

$$= \frac{D}{2} \log \sigma^2 + \frac{D}{2} - \frac{1}{2} \|x-G(z)\|^2 - \frac{D}{2} \sigma^2 - \frac{1}{2} \|\mu\|^2 + \text{const}$$

Since constants have no impact on the optimization process, we can just omit them.

And since we want to maximize ELBO. So the loss can be written as the negative ELBO

$$\text{Loss} = \frac{1}{2} \|x-G(z)\|^2 - \frac{1}{2} \sum_{j=1}^D (\log \sigma^2 + 1 - \sigma^2 - \mu_j^2) \\ = \text{reconstruction_loss} + \text{KL-loss}$$

The following three cells train the model. You can just run them. It may take a while to run on your laptop.

```
In [16]: X_train, X_val = train_test_split(data, test_size=0.2, random_state=365)
```

```
In [17]: batch_size = 64
# autoencoder.compile(loss=get_loss(distribution_mean, distribution_variance, factor = 100,
#                                     batch_size = batch_size), optimizer='adam')
autoencoder.compile(optimizer="adam", loss=reconstruction_loss)

autoencoder.summary()
```

Model: "autoencoder_with_kl"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 45, 45, 3)	0	–
conv2d (Conv2D)	(None, 41, 41, 64)	4,864	input_layer[0][0]
max_pooling2d (MaxPooling2D)	(None, 20, 20, 64)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 18, 18, 64)	36,928	max_pooling2d[0]...
max_pooling2d_1 (MaxPooling2D)	(None, 9, 9, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 7, 7, 32)	18,464	max_pooling2d_1[...
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 32)	0	conv2d_2[0][0]
flatten (Flatten)	(None, 288)	0	max_pooling2d_2[...
functional (Functional)	(None, 100)	118,056	input_layer[0][0]
variational_mean (Dense)	(None, 100)	28,900	flatten[0][0]
variational_log_va... (Dense)	(None, 100)	28,900	flatten[0][0]
functional_1 (Functional)	(None, 45, 45, 3)	90,675	functional[0][0]
kl_loss (KLLoss)	[(None, 100), (None, 100)]	0	variational_mean... variational_log_...
with_kl (Lambda)	(None, 45, 45, 3)	0	functional_1[0][... kl_loss[0][0], kl_loss[0][1]

Total params: 208,731 (815.36 KB)
Trainable params: 208,731 (815.36 KB)
Non-trainable params: 0 (0.00 B)

```
In [18]: autoencoder.fit(X_train, X_train, epochs=50, batch_size=64, validation_data=(X_val, X_val))
```

Epoch 1/50
165/165 ————— 35s 195ms/step – loss: 108.8332 – val_loss: 71.0791
Epoch 2/50
165/165 ————— 23s 142ms/step – loss: 65.1133 – val_loss: 60.3446
Epoch 3/50
165/165 ————— 21s 130ms/step – loss: 58.4044 – val_loss: 56.4822
Epoch 4/50
165/165 ————— 22s 131ms/step – loss: 55.7784 – val_loss: 56.5492
Epoch 5/50
165/165 ————— 24s 144ms/step – loss: 53.8797 – val_loss: 52.1220
Epoch 6/50
165/165 ————— 28s 166ms/step – loss: 51.3393 – val_loss: 50.1095
Epoch 7/50
165/165 ————— 25s 154ms/step – loss: 49.3629 – val_loss: 49.4646
Epoch 8/50
165/165 ————— 23s 139ms/step – loss: 48.4944 – val_loss: 47.6736
Epoch 9/50
165/165 ————— 25s 152ms/step – loss: 47.3959 – val_loss: 46.8746
Epoch 10/50
165/165 ————— 25s 154ms/step – loss: 46.8033 – val_loss: 46.5947
Epoch 11/50
165/165 ————— 22s 135ms/step – loss: 45.8579 – val_loss: 45.3694
Epoch 12/50
165/165 ————— 22s 132ms/step – loss: 45.1829 – val_loss: 44.2963
Epoch 13/50
165/165 ————— 24s 143ms/step – loss: 44.6813 – val_loss: 43.9328
Epoch 14/50
165/165 ————— 21s 128ms/step – loss: 44.1417 – val_loss: 44.2478
Epoch 15/50
165/165 ————— 25s 152ms/step – loss: 43.8882 – val_loss: 43.3570
Epoch 16/50
165/165 ————— 25s 148ms/step – loss: 43.6131 – val_loss: 43.5834
Epoch 17/50

```

165/165 ————— 24s 145ms/step - loss: 43.6980 - val_loss: 43.0711
Epoch 18/50
165/165 ————— 24s 147ms/step - loss: 43.2190 - val_loss: 42.8289
Epoch 19/50
165/165 ————— 25s 149ms/step - loss: 43.0238 - val_loss: 42.9328
Epoch 20/50
165/165 ————— 24s 146ms/step - loss: 42.9552 - val_loss: 43.1851
Epoch 21/50
165/165 ————— 26s 159ms/step - loss: 43.0415 - val_loss: 42.4648
Epoch 22/50
165/165 ————— 25s 149ms/step - loss: 42.7804 - val_loss: 42.4376
Epoch 23/50
165/165 ————— 24s 147ms/step - loss: 42.7362 - val_loss: 42.4325
Epoch 24/50
165/165 ————— 24s 145ms/step - loss: 42.5668 - val_loss: 42.1636
Epoch 25/50
165/165 ————— 24s 145ms/step - loss: 42.5392 - val_loss: 42.2155
Epoch 26/50
165/165 ————— 24s 147ms/step - loss: 42.4091 - val_loss: 42.2067
Epoch 27/50
165/165 ————— 24s 146ms/step - loss: 42.3632 - val_loss: 43.3832
Epoch 28/50
165/165 ————— 24s 148ms/step - loss: 42.3215 - val_loss: 42.3867
Epoch 29/50
165/165 ————— 24s 145ms/step - loss: 42.1584 - val_loss: 42.5914
Epoch 30/50
165/165 ————— 24s 146ms/step - loss: 42.1661 - val_loss: 41.8397
Epoch 31/50
165/165 ————— 25s 149ms/step - loss: 41.9561 - val_loss: 41.6783
Epoch 32/50
165/165 ————— 24s 146ms/step - loss: 41.9127 - val_loss: 41.7095
Epoch 33/50
165/165 ————— 23s 142ms/step - loss: 41.8159 - val_loss: 42.4661
Epoch 34/50
165/165 ————— 25s 151ms/step - loss: 41.8033 - val_loss: 41.5962
Epoch 35/50
165/165 ————— 24s 145ms/step - loss: 41.6950 - val_loss: 41.4522
Epoch 36/50
165/165 ————— 25s 150ms/step - loss: 41.6367 - val_loss: 41.6086
Epoch 37/50
165/165 ————— 24s 145ms/step - loss: 41.7287 - val_loss: 41.8375
Epoch 38/50
165/165 ————— 26s 156ms/step - loss: 41.4224 - val_loss: 41.4703
Epoch 39/50
165/165 ————— 24s 148ms/step - loss: 41.4120 - val_loss: 42.1664
Epoch 40/50
165/165 ————— 24s 145ms/step - loss: 41.4478 - val_loss: 42.2404
Epoch 41/50
165/165 ————— 24s 147ms/step - loss: 41.3480 - val_loss: 42.5395
Epoch 42/50
165/165 ————— 25s 151ms/step - loss: 41.2937 - val_loss: 41.4931
Epoch 43/50
165/165 ————— 24s 142ms/step - loss: 41.3455 - val_loss: 41.0472
Epoch 44/50
165/165 ————— 25s 148ms/step - loss: 41.2176 - val_loss: 41.0351
Epoch 45/50
165/165 ————— 24s 146ms/step - loss: 41.1123 - val_loss: 41.3226
Epoch 46/50
165/165 ————— 24s 145ms/step - loss: 41.2207 - val_loss: 41.6598
Epoch 47/50
165/165 ————— 24s 147ms/step - loss: 41.0883 - val_loss: 40.9495
Epoch 48/50
165/165 ————— 24s 145ms/step - loss: 41.0171 - val_loss: 41.0616
Epoch 49/50
165/165 ————— 24s 147ms/step - loss: 40.9311 - val_loss: 40.8493
Epoch 50/50
165/165 ————— 24s 146ms/step - loss: 40.9968 - val_loss: 41.4592

```

Out[18]: <keras.src.callbacks.history.History at 0x1665bd2b0>

```

In [30]: # Save your trained model (for later use)
autoencoder.save("./trained_autoencoder.keras")
encoder_model.save("./trained_encoder.keras")
decoder_model.save("./trained_decoder.keras")

```

```

1/1 ————— 0s 263ms/step
1/1 ————— 0s 263ms/step

```

If you experience issues while training the models above or if the process is taking too long, you may also use our pre-trained versions of the VAE, encoder, and decoder, which are available on Canvas.

No points will be taken off if you decide to use the pre-trained models, but we do recommend trying out the training process

above. In case you encounter any issues when loading in the models, please post your questions on Ed discussion.

```
In [ ]: # Load Pre-trained Model

# batch_size = 64
# # autoencoder_ = tensorflow.keras.models.load_model("./trained_autoencoder.h5", compile=False, safe_mode=False)
# autoencoder.compile(loss=get_loss(distribution_mean, distribution_variance, factor = 100, batch_size = batch_size))

# # encoder_model_ = tensorflow.keras.models.load_model("./trained_encoder.h5", compile=False, safe_mode=False)
# encoder_model.compile(loss=get_loss(distribution_mean, distribution_variance, factor = 100, batch_size = batch_size))

# # decoder_model_ = tensorflow.keras.models.load_model("./trained_decoder.h5", compile=False, safe_mode=False)
# decoder_model.compile(loss=get_loss(distribution_mean, distribution_variance, factor = 100, batch_size = batch_size))
```

1.2 Reconstructing faces (3 points)

The following cell encodes and reconstructs 16 random faces from the validation set with the trained VAE. Run the cell and comment on the reconstructed faces. (3 points)

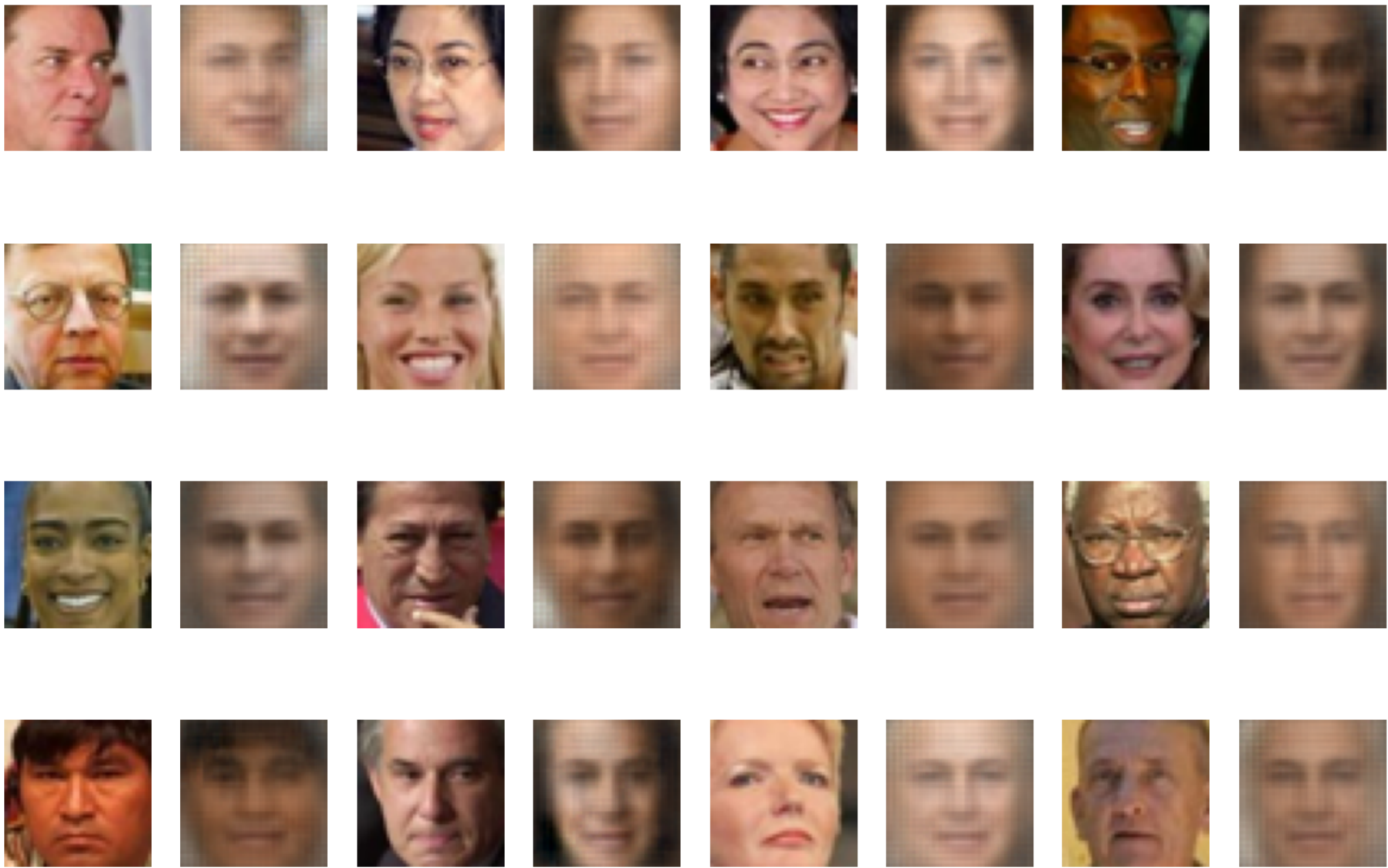
- Do the reconstructed faces resemble the original images? How are they similar/different?
- Are there any faces that are reconstructed better or worse than the others? Can you think of why?
- Comment on any other aspects of your findings that are interesting to you.

```
In [20]: # sample_index = random.sample(range(1, len(X_val)), 16)
sample_index = range(16)

fig, axs = plt.subplots(4, 8)
fig.set_figheight(10)
fig.set_figwidth(15)

for i in range(4):
    for j in range(4):
        axs[i, 2*j].imshow(X_val[sample_index[4*i+j], :, :, :])
        axs[i, 2*j].axis('off')
        axs[i, 2*j+1].imshow(np.clip(autoencoder.predict(np.array([X_val[sample_index[4*i+j], :, :, :])))[0], 0, 1))
        axs[i, 2*j+1].axis('off')
```

1/1 _____ 0s 226ms/step
1/1 _____ 0s 16ms/step
1/1 _____ 0s 18ms/step
1/1 _____ 0s 17ms/step
1/1 _____ 0s 17ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 18ms/step
1/1 _____ 0s 19ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 18ms/step
1/1 _____ 0s 18ms/step
1/1 _____ 0s 17ms/step
1/1 _____ 0s 17ms/step
1/1 _____ 0s 19ms/step



- Do the reconstructed faces resemble the original images? How are they similar/different?
 - The reconstructed faces likely capture the overall structure: general face shape, face direction, and positions of eyes, nose, mouth.
 - But finer details are different: The wrinkles, glasses, shape of eyes, nose, mouth and skin color.
- Are there any faces that are reconstructed better or worse than the others? Can you think of why?
 - I think (2, 3) and (4, 3) are reconstructed better while (2, 1) (4, 4) are worse.
 - This might be because that faces with features common in the training set are usually reconstructed more accurately. The better ones are just common and average faces with no special expressions or features, while the worse ones are people with special features which might not be well captured in the model.
- Comment on any other aspects of your findings that are interesting to you.
 - VAEs inherently tend to produce blurrier images.
 - The features tend to go average.

1.3 Visualizing the latent space (10 points)

In `vae_demo` from class, the MNIST digits were generated from a two-dimensional latent space. In the current model, the latent space has more than two dimensions, so to visualize it we need to use a dimensionality reduction technique. (If you are not familiar with PCA, please refer to the material for Week 7 of [iML](#).)

In this problem, you will first implement the function `LatentSpace_2D` . (6 points)

1. Calculate the latent space encodings for two sets of faces that are different in one attribute, e.g. smile vs. no smile.
2. Use PCA to reduce the dimension of the latent space codes to two.
3. Visualize the latent space after dimensionality reduction with a scatter plot. Clearly color-code and label the two different groups.

Here is an example using `smile_data` and `no_smile_data`.

Visualize the latent space for at least three pairs of face groups including smile vs. no smile. Comment on how the scatter plots look.

- Are the two groups separable in the two-dimensional latent space? Is this what you expected? Why or why not? (2 points)
- How do the plots for the three different attributes differ from each other? (2 points)

```
In [21]: def LatentSpace_2D(encoder_model, data1, label1, data2, label2):
# Your code here
latent_group1 = encoder_model.predict(data1)
latent_group2 = encoder_model.predict(data2)

latent_all = np.vstack([latent_group1, latent_group2])
```

```
pca = PCA(n_components=2)
latent_2d = pca.fit_transform(latent_all)

latent1_2d = latent_2d[:len(latent_group1)]
latent2_2d = latent_2d[len(latent_group1):]

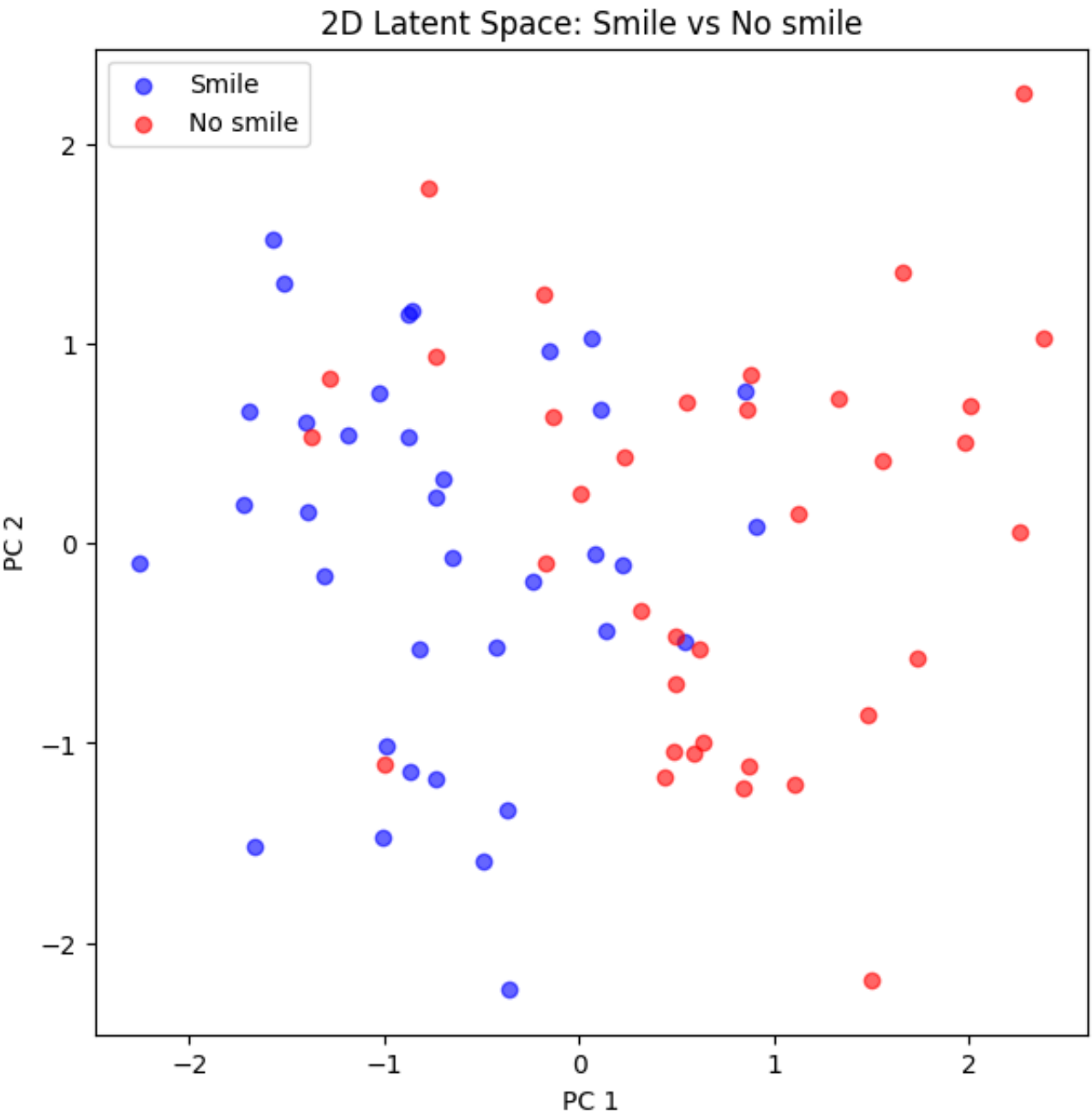
fig, ax = plt.subplots(figsize=(7,7))
ax.scatter(latent1_2d[:,0], latent1_2d[:,1], c='blue', label=label1, alpha=0.6)
ax.scatter(latent2_2d[:,0], latent2_2d[:,1], c='red', label=label2, alpha=0.6)
ax.set_xlabel("PC 1")
ax.set_ylabel("PC 2")
ax.set_title(f"2D Latent Space: {label1} vs {label2}")
ax.legend()
plt.show()
```

```
In [56]: encoder_mean = tensorflow.keras.Model(encoder_model.input, distribution_mean)
LatentSpace_2D(encoder_mean, smile_data, 'Smile', no_smile_data, 'No smile')
```

1/2 ————— 0s 72ms/stepWARNING:tensorflow:5 out of the last 27 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x108bf0e00> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

2/2 ————— 0s 70ms/step

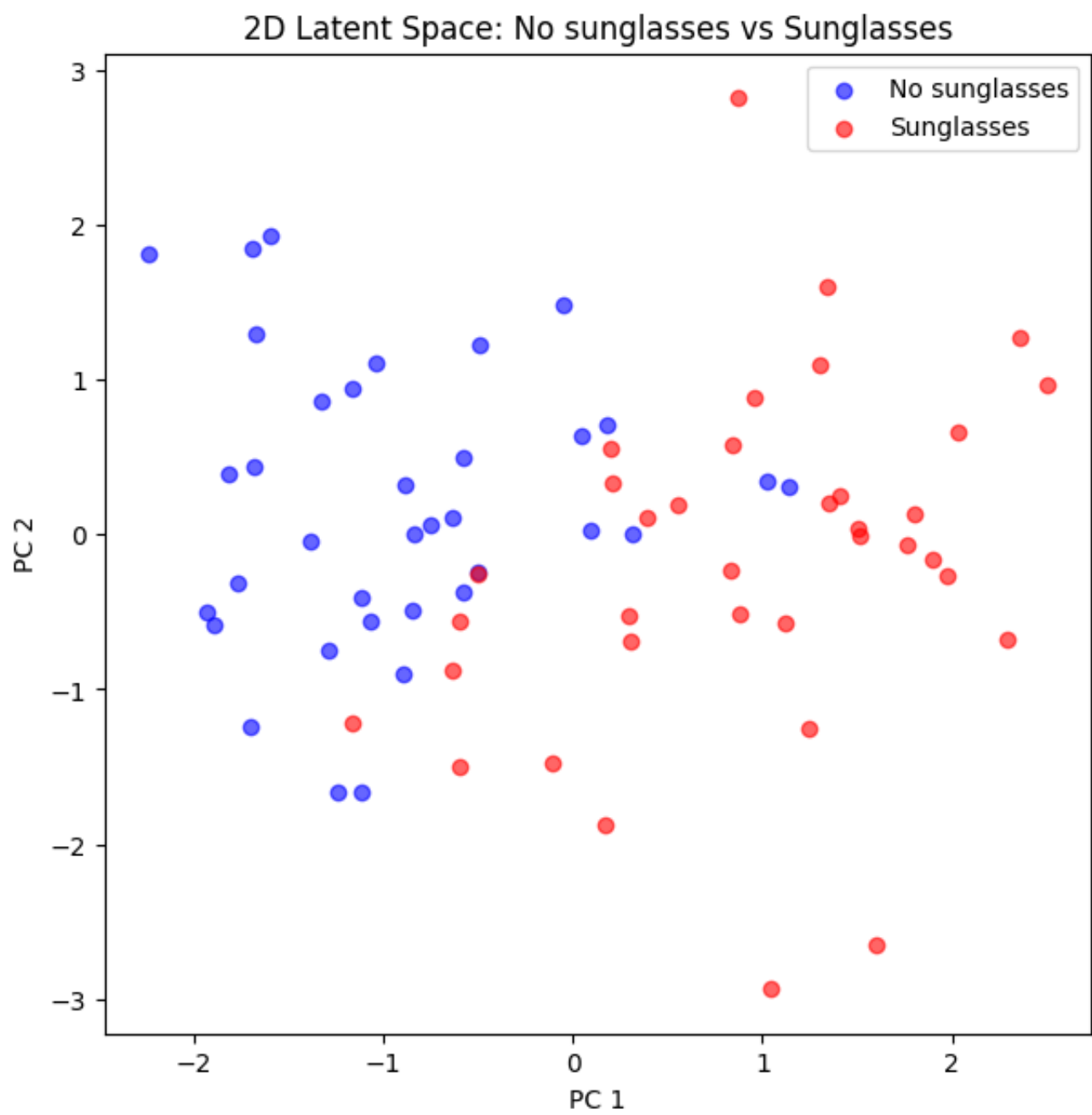
2/2 ————— 0s 13ms/step



```
In [57]: LatentSpace_2D(encoder_mean,smile_data,'No sunglasses',sunglasses_data,'Sunglasses')
```

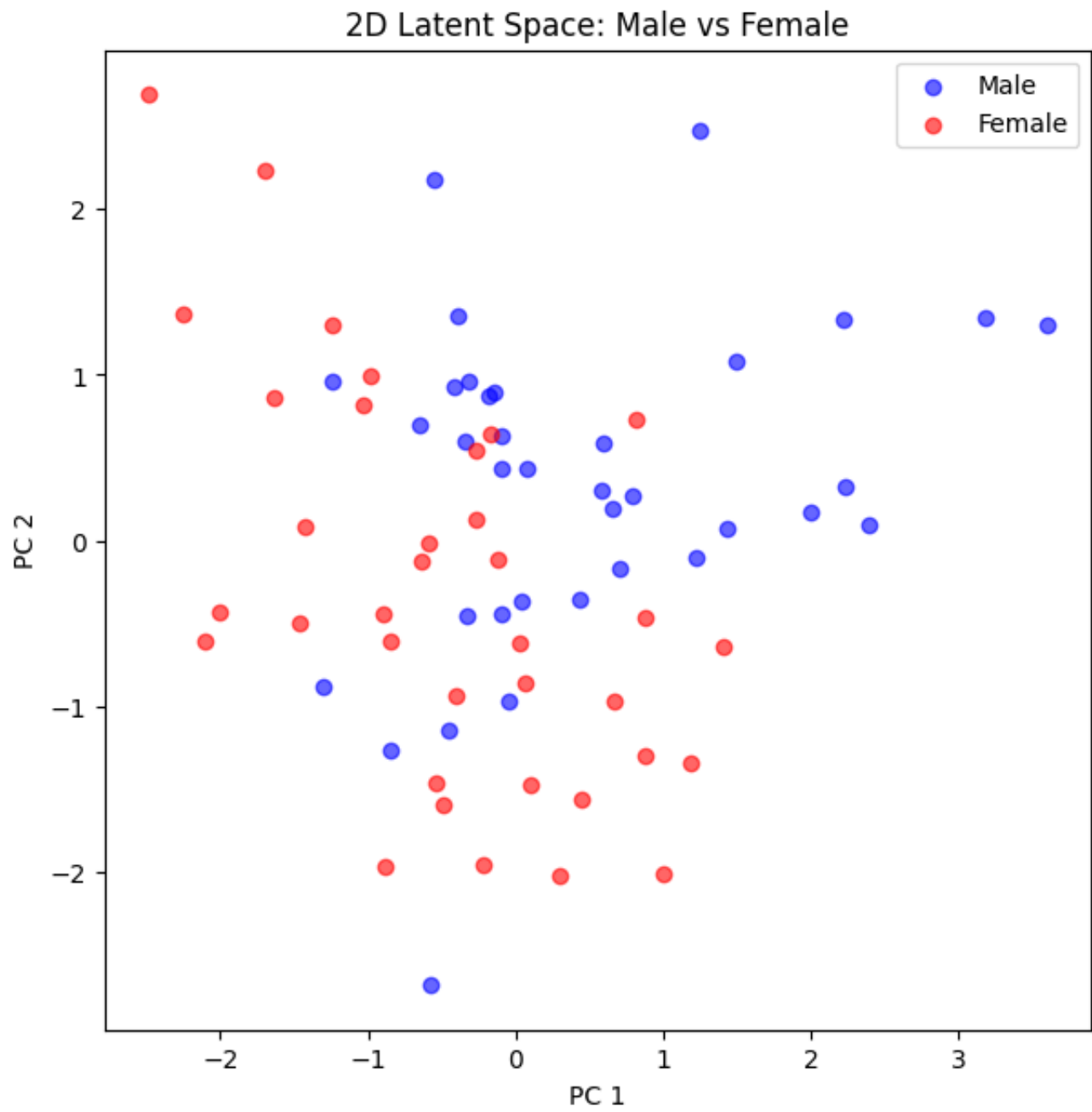
2/2 ————— 0s 69ms/step

2/2 ————— 0s 57ms/step



```
In [58]: LatentSpace_2D(encoder_mean,male_data,'Male',female_data,'Female')
```

2/2 ————— 0s 58ms/step
2/2 ————— 0s 52ms/step



- Are the two groups separable in the two-dimensional latent space? Is this what you expected? Why or why not? (2 points)

- The two groups are mostly well separated for the smile and glasses attributes, but not so well for the gender attributes.
- This is what I expected. I think it's because the encoder learns to map inputs with similar high-level features in the latent space. However, since the VAE also optimizes the KL divergence term that enforces a Gaussian prior, it's impossible to reach a complete separation.
- How do the plots for the three different attributes differ from each other? (2 points)
 - Genders are harder to separate while smiles and glasses are better separated.
 - This may be because that the smile and glasses attributes are distinct between different faces to be categorized, but the gender attributes does not have much distinction, so it's not easy to separate them.

1.4 Morphing between faces (4 points)

Morph at least 5 pairs of faces with the function `morphBetweenImages` and comment on what you observe.

- Briefly explain how the morphing works. (2 points)
- Do the generated faces look like what you expected? Does any of the pairs work better than the others? If so, what kind of image pairs work better? (2 points)

```
In [38]: # Don't change the function
def morphBetweenImages(img1, img2, num_of_morphs):
    alpha = np.linspace(0,1,num_of_morphs)
    z1 = encoder_model.predict(np.array([img1]))
    z2 = encoder_model.predict(np.array([img2]))
    fig = plt.figure(figsize=(30,5))

    ax = fig.add_subplot(1, num_of_morphs+2, 1)
    ax.imshow(img1)
    ax.axis('off')
    ax.set_title(loc='center', label='original image 1',fontsize=10)

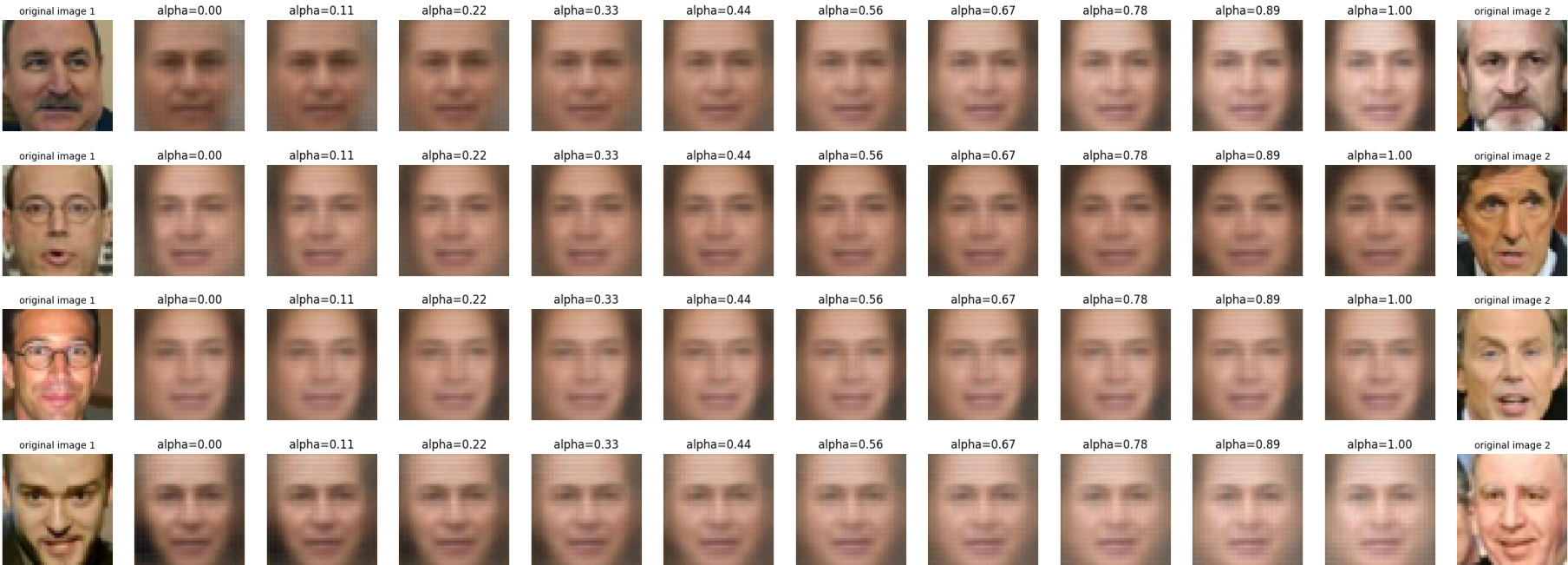
    for i in range(num_of_morphs):
        z = z1*(1-alpha[i]) + z2*alpha[i]
        new_img = decoder_model.predict(z)

        ax = fig.add_subplot(1, num_of_morphs+2, i+2)
        ax.imshow(np.clip(new_img.squeeze(),0,1))
        ax.axis('off')
        ax.set_title(loc='center', label='alpha={:.2f}'.format(alpha[i]))

    ax = fig.add_subplot(1, num_of_morphs+2, num_of_morphs+2)
    ax.imshow(img2)
    ax.axis('off')
    ax.set_title(loc='center', label='original image 2',fontsize=10)
    return
```

```
In [49]: for i in range(5):
    sample_index = random.sample(range(1, len(data)), 2)
    morphBetweenImages(data[sample_index[0]],data[sample_index[1]],10)
```

1/1		0s	36ms/step
1/1		0s	33ms/step
1/1		0s	36ms/step
1/1		0s	32ms/step
1/1		0s	27ms/step
1/1		0s	32ms/step
1/1		0s	31ms/step
1/1		0s	25ms/step
1/1		0s	24ms/step
1/1		0s	23ms/step
1/1		0s	24ms/step
1/1		0s	24ms/step
1/1		0s	24ms/step
1/1		0s	33ms/step
1/1		0s	27ms/step
1/1		0s	27ms/step
1/1		0s	27ms/step
1/1		0s	27ms/step
1/1		0s	25ms/step
1/1		0s	23ms/step
1/1		0s	24ms/step
1/1		0s	23ms/step
1/1		0s	25ms/step
1/1		0s	24ms/step
1/1		0s	25ms/step
1/1		0s	25ms/step
1/1		0s	24ms/step
1/1		0s	24ms/step
1/1		0s	25ms/step
1/1		0s	28ms/step
1/1		0s	26ms/step
1/1		0s	38ms/step
1/1		0s	33ms/step
1/1		0s	39ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	33ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	36ms/step
1/1		0s	37ms/step
1/1		0s	35ms/step
1/1		0s	30ms/step
1/1		0s	27ms/step
1/1		0s	31ms/step
1/1		0s	35ms/step
1/1		0s	39ms/step
1/1		0s	69ms/step
1/1		0s	42ms/step
1/1		0s	32ms/step
1/1		0s	27ms/step
1/1		0s	27ms/step
1/1		0s	24ms/step
1/1		0s	24ms/step
1/1		0s	23ms/step
1/1		0s	25ms/step
1/1		0s	23ms/step
1/1		0s	23ms/step
1/1		0s	23ms/step





- Briefly explain how the morphing works.
 - Each face is first encoded into the latent space using the `encoder_model` and get vector z_1 for `img1` and vector z_2 for `img2`.
 - Then morph linearly interpolates between these latent vectors: $z_\alpha = (1 - \alpha)z_1 + \alpha z_2$, $\alpha \in [0, 1]$
 - Each interpolated latent vector z_α is decoded into an image using the `decoder_model`.
 - The result is a smooth transition from the first face to the second in latent space.
- Do the generated faces look like what you expected? Does any of the pairs work better than the others? If so, what kind of image pairs work better?
 - Yes, it's like what I expected.
 - I think they all work well except for the 4th line. Maybe it's because the faces in the 4th line have different directions, which makes the transition between them a little difficult to do.
 - Maybe images with similar features can be morphed better, while faces with too much distinction may not have good morphing.

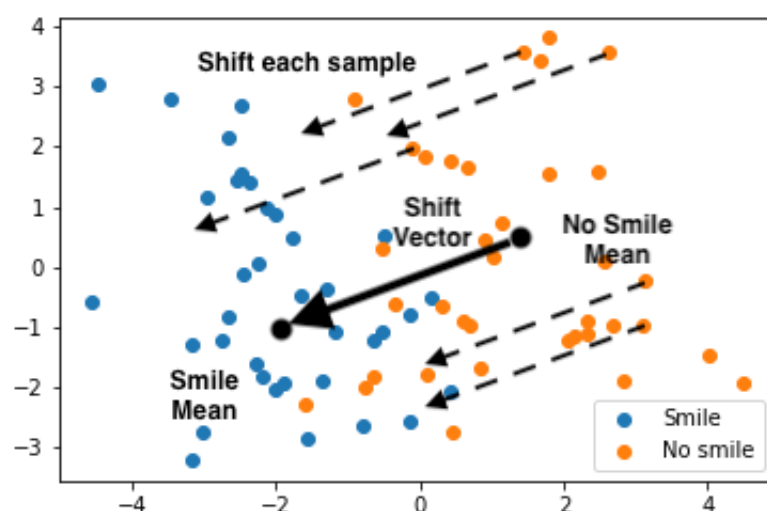
1.5 Attribute shift (10 points)

In 1.3, we've seen that faces with the same attributes form clusters in the latent space. In this problem, you will implement a function `AttributeShift` to change one attribute of the faces.

First implement the function `AttributeShift`. (5 points)

- Calculate the latent space codes for two sets of faces that are different in one attribute, e.g. smile vs. no smile.
- Calculate the mean latent space code for each group.
- Get the attribute shifting vector by taking the difference between the two codes.
- Perform attribute shift by adding the attribute shifting vector to the latent space code of the faces you want to modify.
- Generate the image using the new latent space codes.

Here is a diagram demonstrating the shift in the latent space. Please note that the two-dimensional latent space is just for demonstration purpose. You should *not* use PCA in this problem. Instead, use the original latent space.



Perform attribute shift on at three attributes including smile. Comment on the faces with shifted attributes. (5 points)

- Do the generated faces look like what you expected? If not, can you think of some possible reasons?
- Do the faces with new attributes resemble the original faces? If not, can you think of some possible reasons?
- Which of the attribute shift is more successful? What are some possible reasons?
- Comment on any other aspects of your findings that are interesting to you.

```
In [60]: # Don't change this helper function!
def PlotAttributeShift(data2, pic_output):
    sample_index = random.sample(range(1, len(data2)), 16)

    fig, axs = plt.subplots(4, 8)
    fig.set_figheight(10)
    fig.set_figwidth(15)

    for i in range(4):
        for j in range(4):
            axs[i, 2*j].imshow(data2[sample_index[4*i+j], :, :, :])
            axs[i, 2*j].axis('off')
            axs[i, 2*j+1].imshow(np.clip(pic_output[sample_index[4*i+j]], 0, 1))
```

```
    axs[i, 2*j+1].axis('off')
```

```
In [61]: def AttributeShift(encoder_model,decoder_model,data1,data2):
        latent1 = encoder_model.predict(data1)
        latent2 = encoder_model.predict(data2)

        mean_latent1 = np.mean(latent1, axis=0)
        mean_latent2 = np.mean(latent2, axis=0)

        attribute_shift_vector = mean_latent2 - mean_latent1

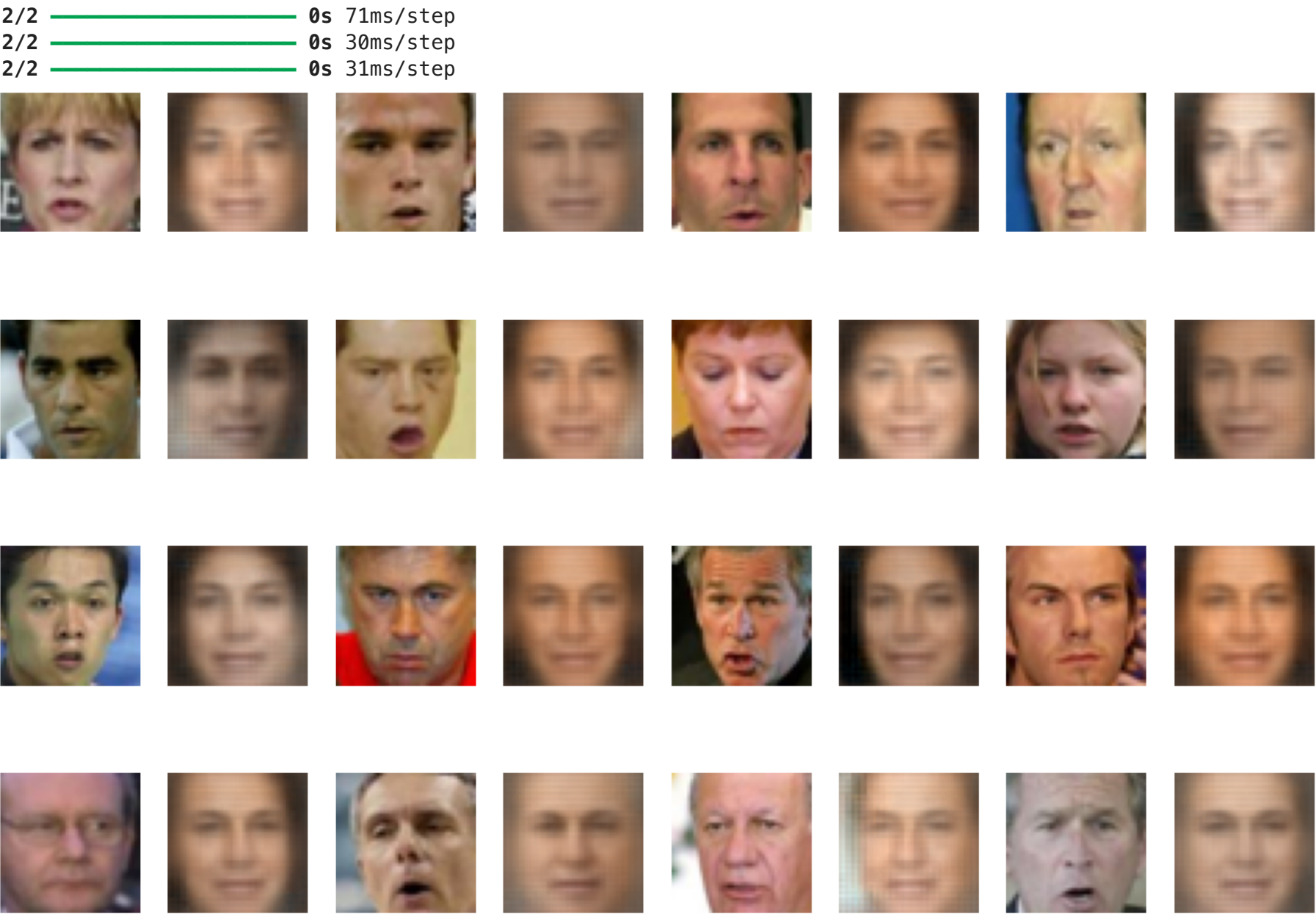
        shifted_latents = latent1 + attribute_shift_vector

        pic_output = decoder_model.predict(shifted_latents)

        return pic_output
```

The dataset has the following attributes: ['Male', 'Asian', 'White', 'Black', 'Baby', 'Child', 'Youth', 'Middle Aged', 'Senior', 'Black Hair', 'Blond Hair', 'Brown Hair', 'Bald', 'No Eyewear', 'Eyeglasses', 'Sunglasses', 'Mustache', 'Smiling', 'Frowning', 'Chubby', 'Blurry', 'Harsh Lighting', 'Flash', 'Soft Lighting', 'Outdoor', 'Curly Hair', 'Wavy Hair', 'Straight Hair', 'Receding Hairline', 'Bangs', 'Sideburns', 'Fully Visible Forehead', 'Partially Visible Forehead', 'Obstructed Forehead', 'Bushy Eyebrows', 'Arched Eyebrows', 'Narrow Eyes', 'Eyes Open', 'Big Nose', 'Pointy Nose', 'Big Lips', 'Mouth Closed', 'Mouth Slightly Open', 'Mouth Wide Open', 'Teeth Not Visible', 'No Beard', 'Goatee', 'Round Jaw', 'Double Chin', 'Wearing Hat', 'Oval Face', 'Square Face', 'Round Face', 'Color Photo', 'Posed Photo', 'Attractive Man', 'Attractive Woman', 'Indian', 'Gray Hair', 'Bags Under Eyes', 'Heavy Makeup', 'Rosy Cheeks', 'Shiny Skin', 'Pale Skin', '5 o' Clock Shadow', 'Strong Nose-Mouth Lines', 'Wearing Lipstick', 'Flushed Face', 'High Cheekbones', 'Brown Eyes', 'Wearing Earrings', 'Wearing Necktie', 'Wearing Necklace']

```
In [65]: pic_output1 = AttributeShift(encoder_model,decoder_model,no_smile_data,smile_data)
        PlotAttributeShift(no_smile_data,pic_output1)
```



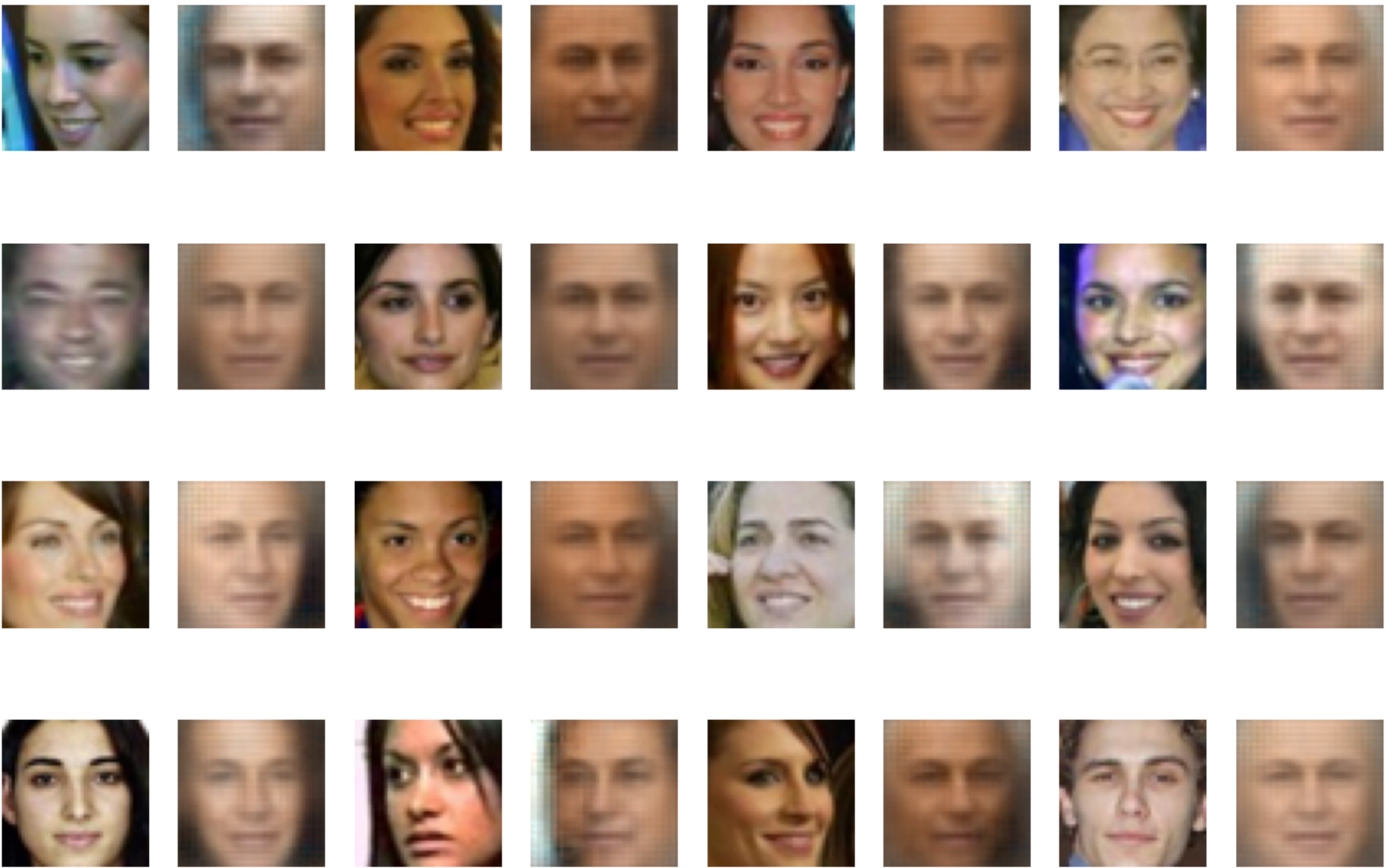
```
In [64]: youth_ids = attrs['Youth'].sort_values(ascending=False).head(36).index.values
        youth_data = data[youth_ids]
        senior_ids = attrs['Senior'].sort_values(ascending=False).head(36).index.values
        senior_data = data[senior_ids]

        pic_output2 = AttributeShift(encoder_model,decoder_model,youth_data,senior_data)
        PlotAttributeShift(youth_data,pic_output2)
```

2/2  0s 30ms/step

2/2  0s 40ms/step

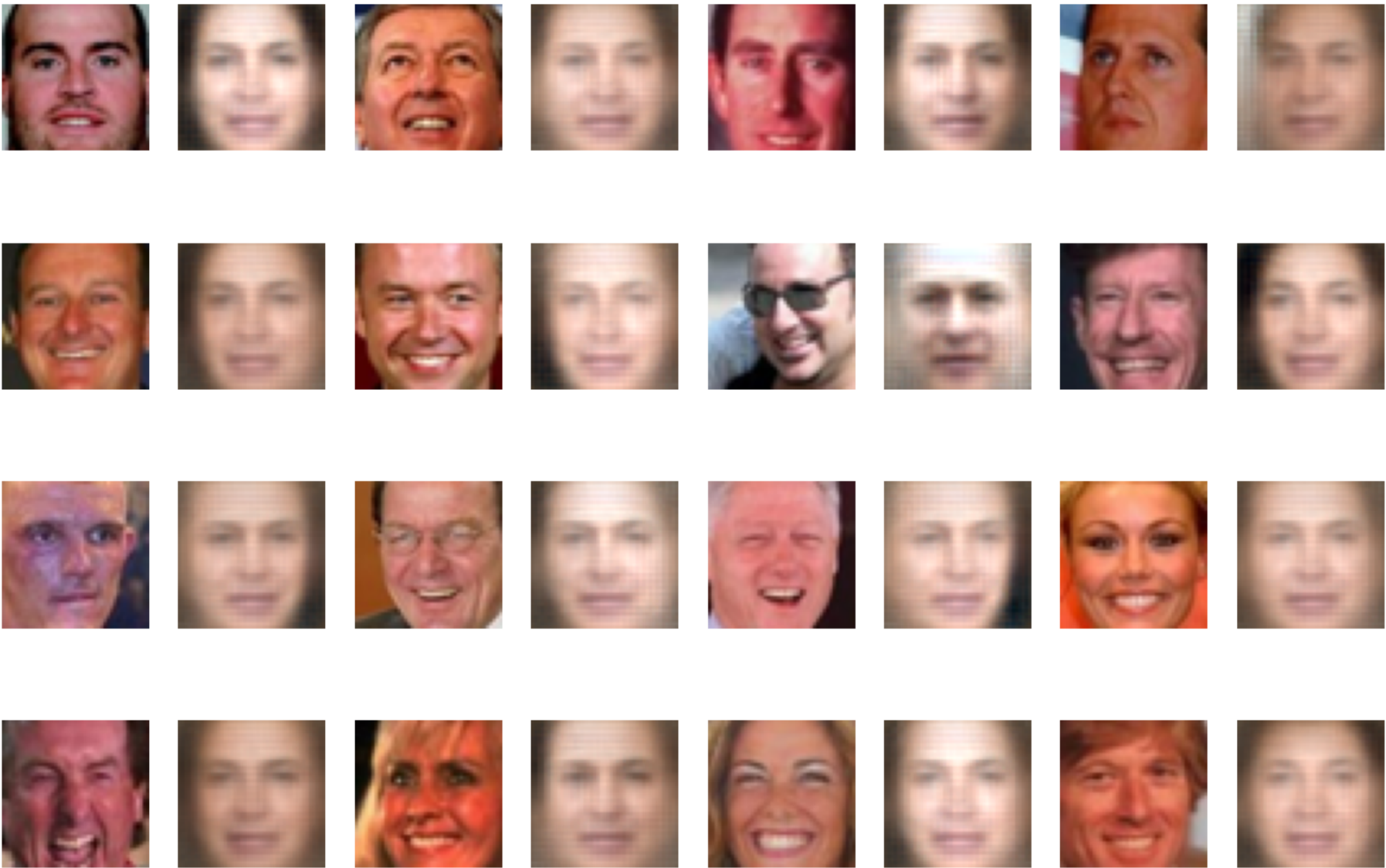
2/2  0s 40ms/step



```
In [72]: pale_skin_ids = attrs['Pale Skin'].sort_values(ascending=False).head(36).index.values
pale_skin_data = data[pale_skin_ids]
flushed_face_ids = attrs['Flushed Face'].sort_values(ascending=False).head(36).index.values
flushed_face_data = data[flushed_face_ids]

pic_output3 = AttributeShift(encoder_model,decoder_model,flushed_face_data,pale_skin_data)
PlotAttributeShift(flushed_face_data,pic_output3)
```

2/2  0s 11ms/step
2/2  0s 10ms/step
2/2  0s 9ms/step



- Do the generated faces look like what you expected? If not, can you think of some possible reasons?
 - I think to some extent, it's what I expected. Like in the unsmiling to smiling case, every face's mouth has been changed to a more smiling shape than the original image; in the youth to senior case, every face looks much more senior than the original one and in the flushed to pale case, every face's skin also turns quite pale.

- Do the faces with new attributes resemble the original faces? If not, can you think of some possible reasons?
 - The face itself has actually been changed. They do not look like themselves that much. I think maybe it's because when we add the attribute shifting vector, we also introduce some other attributes into the original faces, apart from what we want to change, making them more likely to integrate other faces' features.
- Which of the attribute shift is more successful? What are some possible reasons?
 - I think the smile attribute shift appears to be the most successful. It retain the most features from the original faces than the other two, which means the mouth curve is adjusted while most other facial features remain relatively stable.
 - This may be because the “smile” attribute is highly distinct and well-represented in the latent space, allowing the model to learn a clear direction for that change. In contrast, attributes like age or skin tone are more complex and correlated with other features like wrinkles, shape, lighting, which makes it harder for the model to extract only the desired transformation.
- Comment on any other aspects of your findings that are interesting to you.
 - I find that some attribute shifts, like the youth to senior case, also subtly change the person's gender or facial structure. This suggests that certain attributes are mixed with each other in the latent space, meaning that the model can't fully separate them when encoding features.

1.6 Generating new faces (3 points)

Variational autoencoders can be used to generate new data; this is why they are generative models. We can sample new data points from the distribution in latent space and reconstruct new, fake faces based on them.

To draw a sample close to an existing sample in the latent space, we can add a scaled random sample from the normal distribution to the latent space code of an existing sample. The scalar, which we call the `noise_level` is a parameter that we can tune.

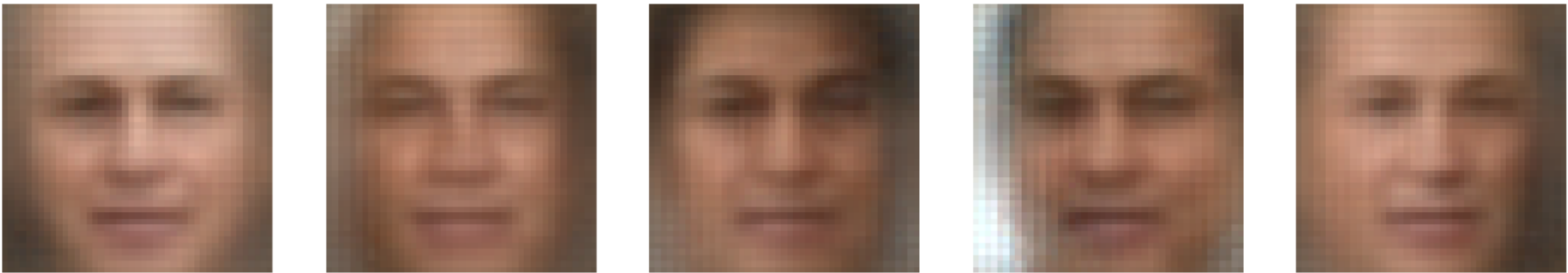
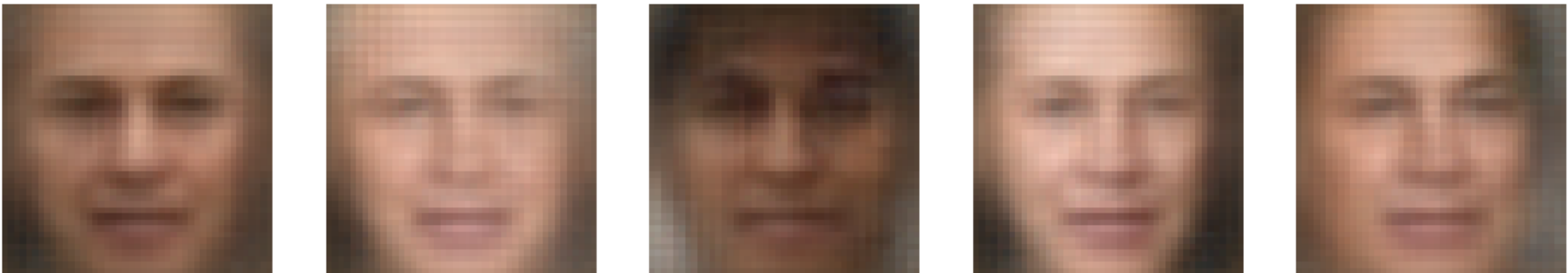
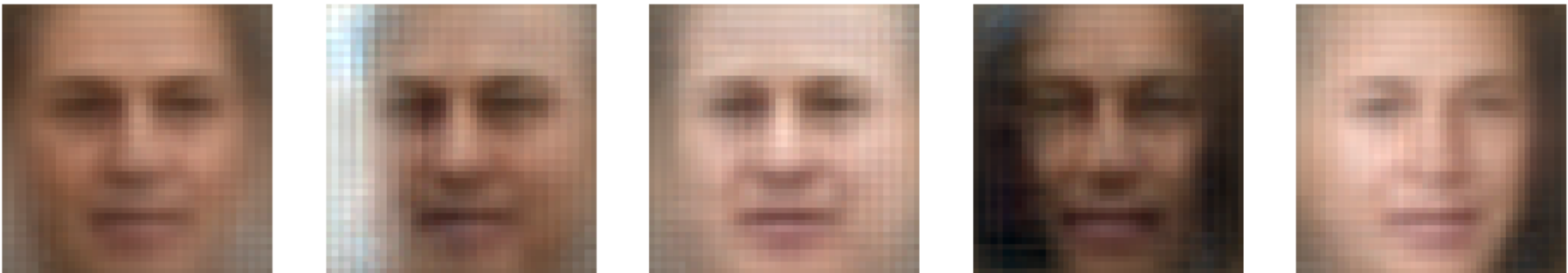
Run `GenerateFaces` with three different values of `noise_level` and comment on the generated faces. (3 points)

- Do the generated images look like faces?
- What happens when the new samples diverge more from the existing samples? What is a possible reason?

```
In [77]: def GenerateFaces(data, LATENT_SPACE_SIZE, noise_level):
sample_index = random.sample(range(1, len(data)), 15)
latent_space = noise_level*np.random.normal(size=(15,LATENT_SPACE_SIZE))+encoder_model.predict(data[san
generated_image = decoder_model.predict(latent_space)
fig = plt.figure(figsize=(15,10))
for i in range(15):
    ax = fig.add_subplot(3, 5, i+1)
    ax.imshow(np.clip(generated_image[i, :, :, :],0,1))
    ax.axis('off')
```

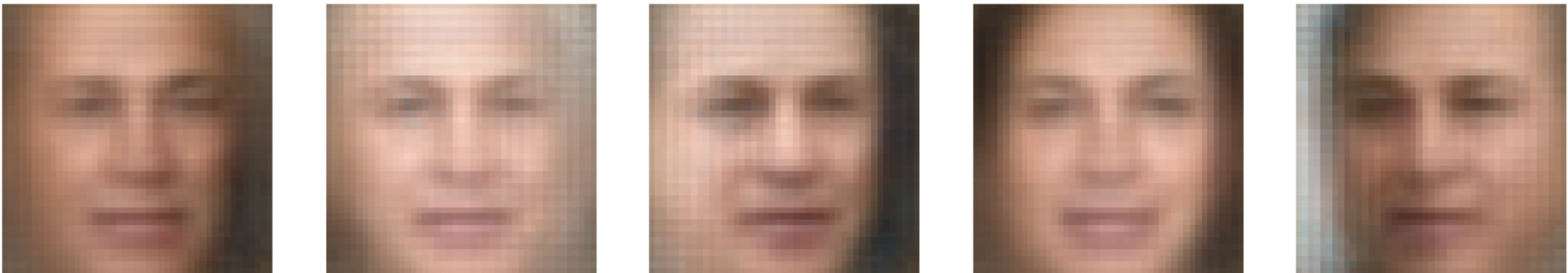
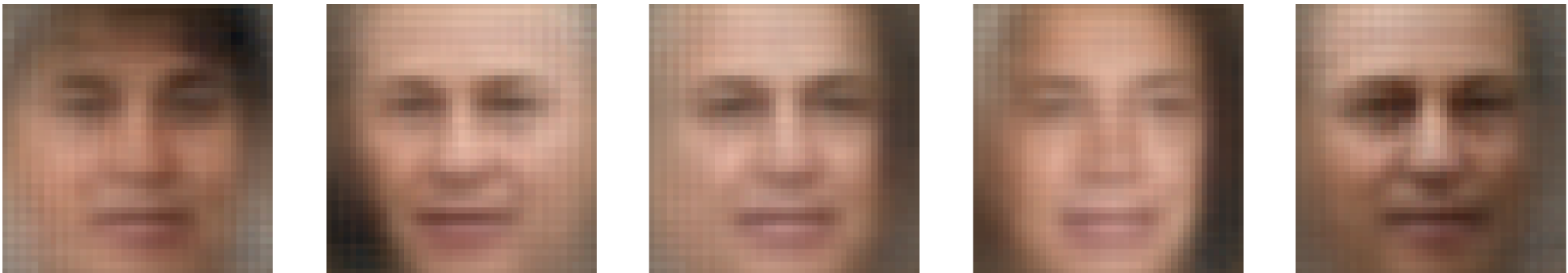
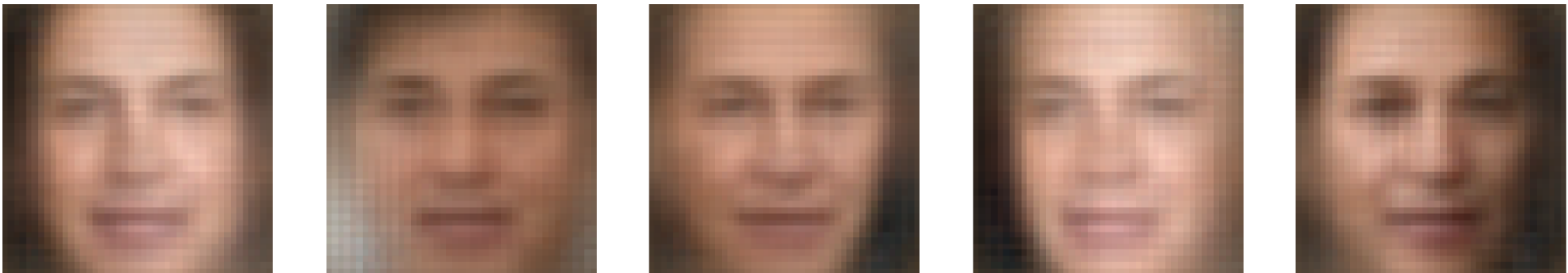
```
In [78]: GenerateFaces(data,LATENT_SPACE_SIZE,0.1)
```

```
1/1 ————— 0s 22ms/step
1/1 ————— 0s 19ms/step
```



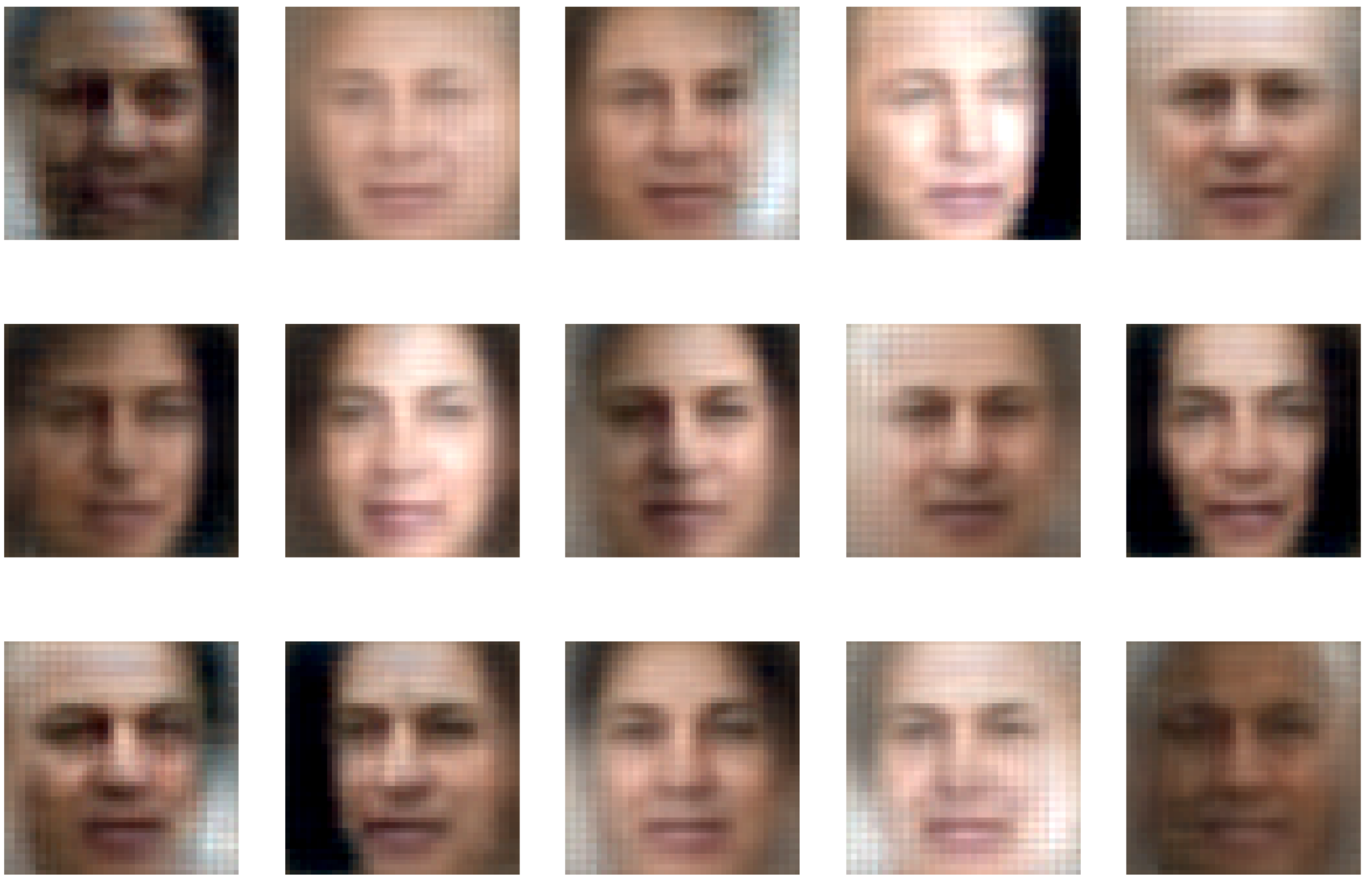
```
In [79]: GenerateFaces(data,LATENT_SPACE_SIZE,0.5)
```

1/1 ————— 0s 24ms/step
1/1 ————— 0s 21ms/step



```
In [80]: GenerateFaces(data,LATENT_SPACE_SIZE,1)
```

1/1 ————— 0s 23ms/step
1/1 ————— 0s 23ms/step



- Do the generated images look like faces?
 - Yes, they look like faces. But as the noise level increases, the generated faces are more blurry and the structures of the faces also become more unclear.
- What happens when the new samples diverge more from the existing samples? What is a possible reason?
 - As the noise level increases, the generated faces are more blurry and the structures of the faces also become more unclear.
 - It might be because larger noise sets the latent vectors farther away from the regions where the VAE has learned meaningful facial representations. In those areas of the latent space, the decoder has not seen enough training data, so it struggles to generate realistic faces.

Optional: Changing the loss function (2 points extra credit)

The variable `factor` is a parameter of the loss function. In this optional problem, you will play with it and think about its effect on the performance of VAE.

Retrain the model with a smaller factor. Repeat 1.2, 1.6 (using `noise_level = 1`) and latent space visualization. Comment on how the reconstruction results, generated new faces and latent space distributions change. (2 points)

- Which model reconstructs the faces better?
- Do the generated faces look different?
- How do the latent variable distributions differ?
- Do the differences make sense? Can you explain what you observed?

```
In [ ]: # set the beta of KLLoss layer to 100
# this will correspondingly make the 'factor' in reconstruction loss to 0.01 in the function given on Ed

encoded_2 = encoder_model(input_data)
decoded_2 = decoder_model(encoded_2)

autoencoder_2 = tensorflow.keras.models.Model(input_data, decoded_2)
# 1) Create the KL node
kl_node_2 = KLLoss(beta=100, name="new_kl_loss")([distribution_mean, distribution_variance])

# 2) Tie the KL node into the forward graph but keep output = decoded
decoded_2 = layers.Lambda(lambda xs: xs[0], name="new_with_kl")([decoded_2, kl_node_2])

# 3) (Re)build the model so the KL loss is included
autoencoder_2 = tensorflow.keras.models.Model(input_data, decoded_2, name="new_autoencoder_with_kl")

autoencoder_2.compile(optimizer="adam", loss=reconstruction_loss)
```

```
In [67]: autoencoder_2.fit(X_train, X_train, epochs=50, batch_size=64, validation_data=(X_val, X_val))
```


Epoch 1/50					
165/165		96s	539ms/step	loss: 90.0111	val_loss: 71.4701
Epoch 2/50					
165/165		57s	348ms/step	loss: 70.9069	val_loss: 70.3545
Epoch 3/50					
165/165		46s	279ms/step	loss: 70.5071	val_loss: 70.4121
Epoch 4/50					
165/165		51s	313ms/step	loss: 70.3786	val_loss: 69.9783
Epoch 5/50					
165/165		55s	333ms/step	loss: 70.3138	val_loss: 70.1186
Epoch 6/50					
165/165		53s	323ms/step	loss: 70.1519	val_loss: 70.0957
Epoch 7/50					
165/165		50s	305ms/step	loss: 70.2035	val_loss: 69.8041
Epoch 8/50					
165/165		54s	330ms/step	loss: 70.1308	val_loss: 69.8157
Epoch 9/50					
165/165		45s	272ms/step	loss: 70.1102	val_loss: 69.8994
Epoch 10/50					
165/165		51s	309ms/step	loss: 70.0316	val_loss: 69.8604
Epoch 11/50					
165/165		50s	302ms/step	loss: 70.0718	val_loss: 69.7821
Epoch 12/50					
165/165		63s	380ms/step	loss: 69.9910	val_loss: 69.8146
Epoch 13/50					
165/165		54s	329ms/step	loss: 70.0803	val_loss: 69.8949
Epoch 14/50					
165/165		52s	319ms/step	loss: 70.0018	val_loss: 69.8052
Epoch 15/50					
165/165		57s	345ms/step	loss: 69.9467	val_loss: 69.7680
Epoch 16/50					
165/165		67s	404ms/step	loss: 69.9743	val_loss: 69.7779
Epoch 17/50					
165/165		47s	283ms/step	loss: 70.0286	val_loss: 69.9581
Epoch 18/50					
165/165		48s	293ms/step	loss: 70.0418	val_loss: 69.6593
Epoch 19/50					
165/165		48s	289ms/step	loss: 69.9797	val_loss: 69.7493
Epoch 20/50					
165/165		61s	373ms/step	loss: 69.8946	val_loss: 69.6528
Epoch 21/50					
165/165		63s	381ms/step	loss: 69.9101	val_loss: 69.6880
Epoch 22/50					
165/165		63s	379ms/step	loss: 69.9192	val_loss: 69.6133
Epoch 23/50					
165/165		46s	279ms/step	loss: 69.8858	val_loss: 69.6824
Epoch 24/50					
165/165		45s	271ms/step	loss: 69.8729	val_loss: 69.7768
Epoch 25/50					
165/165		47s	283ms/step	loss: 69.8976	val_loss: 69.9059
Epoch 26/50					
165/165		57s	348ms/step	loss: 69.8934	val_loss: 69.6050
Epoch 27/50					
165/165		54s	328ms/step	loss: 69.8668	val_loss: 69.7780
Epoch 28/50					
165/165		53s	319ms/step	loss: 69.9025	val_loss: 69.5653
Epoch 29/50					
165/165		53s	321ms/step	loss: 70.0202	val_loss: 69.7523
Epoch 30/50					
165/165		83s	499ms/step	loss: 69.8863	val_loss: 69.6374
Epoch 31/50					
165/165		52s	307ms/step	loss: 69.8859	val_loss: 69.8130
Epoch 32/50					
165/165		641s	4s/step	loss: 69.8682	val_loss: 69.7966
Epoch 33/50					
165/165		512s	3s/step	loss: 69.8342	val_loss: 69.5647
Epoch 34/50					
165/165		70s	416ms/step	loss: 69.8447	val_loss: 69.6121
Epoch 35/50					
165/165		74s	450ms/step	loss: 69.8325	val_loss: 69.6268
Epoch 36/50					
165/165		64s	390ms/step	loss: 69.8411	val_loss: 69.6613
Epoch 37/50					
165/165		54s	329ms/step	loss: 69.8248	val_loss: 69.6528
Epoch 38/50					
165/165		80s	488ms/step	loss: 69.7904	val_loss: 69.5888
Epoch 39/50					
165/165		56s	337ms/step	loss: 69.8147	val_loss: 69.6057
Epoch 40/50					
165/165		80s	486ms/step	loss: 69.8236	val_loss: 69.8175
Epoch 41/50					

165/165 ██████████ 60s 364ms/step - loss: 69.9161 - val_loss: 69.7894
Epoch 42/50
165/165 ██████████ 64s 385ms/step - loss: 69.8088 - val_loss: 69.5500
Epoch 43/50
165/165 ██████████ 76s 462ms/step - loss: 69.8202 - val_loss: 69.5747
Epoch 44/50
165/165 ██████████ 67s 405ms/step - loss: 69.8301 - val_loss: 69.5694
Epoch 45/50
165/165 ██████████ 69s 416ms/step - loss: 69.8093 - val_loss: 69.6408
Epoch 46/50
165/165 ██████████ 58s 352ms/step - loss: 69.8206 - val_loss: 69.6088
Epoch 47/50
165/165 ██████████ 65s 392ms/step - loss: 69.8922 - val_loss: 69.6165
Epoch 48/50
165/165 ██████████ 62s 377ms/step - loss: 69.7669 - val_loss: 69.6154
Epoch 49/50
165/165 ██████████ 58s 350ms/step - loss: 69.7920 - val_loss: 69.5833
Epoch 50/50
165/165 ██████████ 56s 343ms/step - loss: 69.8401 - val_loss: 69.7004

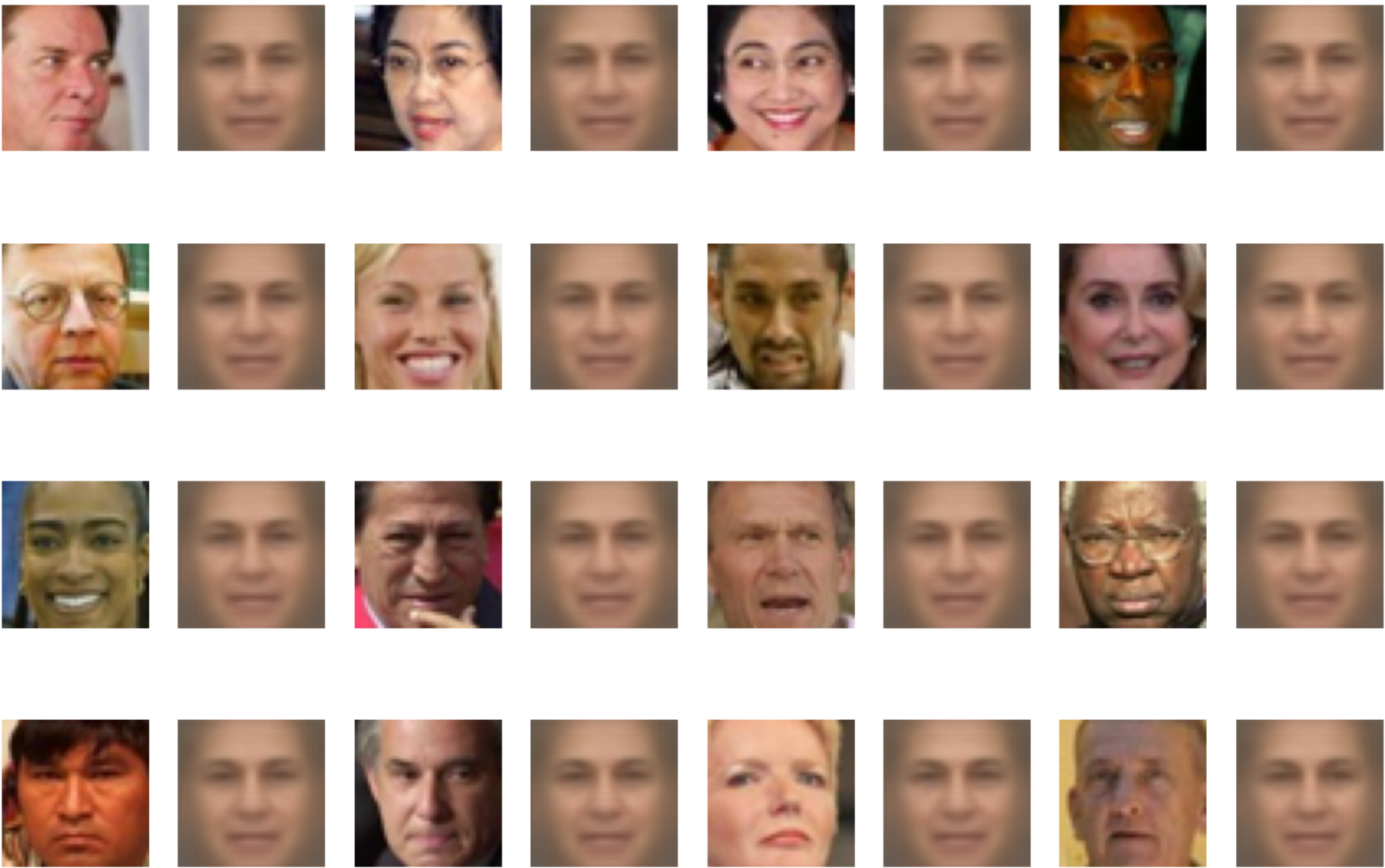
Out[67]: <keras.src.callbacks.history.History at 0x1484623f0>

```
In [68]: # 1.2
# sample_index = random.sample(range(1, len(X_val)), 16)
sample_index = range(16)

fig, axs = plt.subplots(4, 8)
fig.set_figheight(10)
fig.set_figwidth(15)

for i in range(4):
    for j in range(4):
        axs[i, 2*j].imshow(X_val[sample_index[4*i+j], :, :, :])
        axs[i, 2*j].axis('off')
        axs[i, 2*j+1].imshow(np.clip(autoencoder_2.predict(np.array([X_val[sample_index[4*i+j], :, :, :]])),
        axs[i, 2*j+1].axis('off')
```

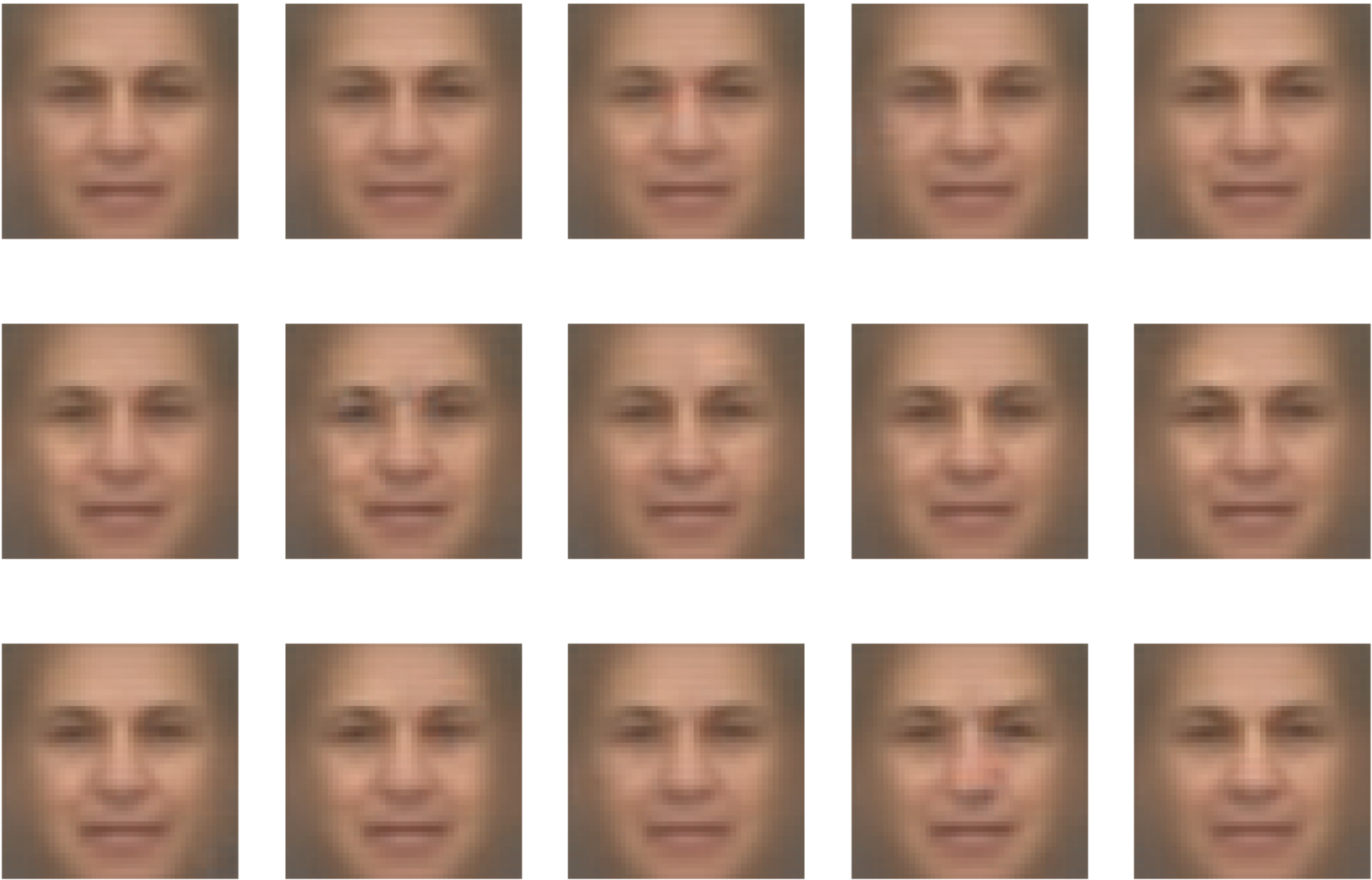
1/1 ██████████ 1s 749ms/step
1/1 ██████████ 0s 40ms/step
1/1 ██████████ 0s 64ms/step
1/1 ██████████ 0s 38ms/step
1/1 ██████████ 0s 34ms/step
1/1 ██████████ 0s 35ms/step
1/1 ██████████ 0s 35ms/step
1/1 ██████████ 0s 50ms/step
1/1 ██████████ 0s 58ms/step
1/1 ██████████ 0s 52ms/step
1/1 ██████████ 0s 56ms/step
1/1 ██████████ 0s 71ms/step
1/1 ██████████ 0s 62ms/step
1/1 ██████████ 0s 59ms/step
1/1 ██████████ 0s 86ms/step
1/1 ██████████ 0s 91ms/step



```
In [69]: # 1.6
def GenerateFaces_(data, LATENT_SPACE_SIZE, noise_level):
    sample_index = random.sample(range(1, len(data)), 15)
    latent_space = noise_level*np.random.normal(size=(15,LATENT_SPACE_SIZE))+encoder_model.predict(data[san
    generated_image = decoder_model.predict(latent_space)
    fig = plt.figure(figsize=(15,10))
    for i in range(15):
        ax = fig.add_subplot(3, 5, i+1)
        ax.imshow(np.clip(generated_image[i, :, :, :],0,1))
        ax.axis('off')

GenerateFaces_(data,LATENT_SPACE_SIZE,1)
```

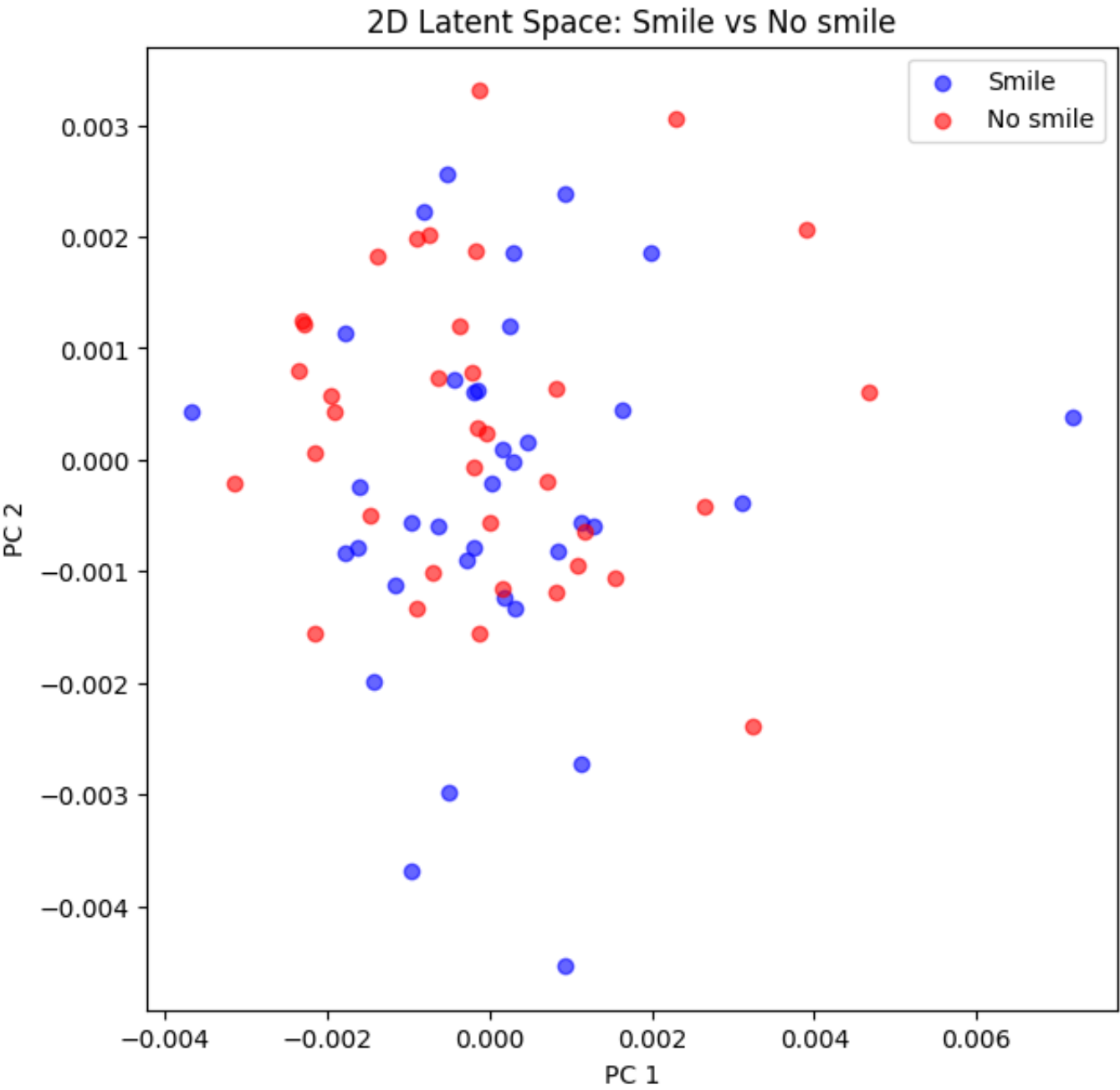
1/1 ————— 0s 29ms/step
1/1 ————— 0s 30ms/step



```
In [70]: # latent space visualization
```

```
encoder_mean2 = tensorflow.keras.Model(encoder_model.input, distribution_mean)
LatentSpace_2D(encoder_mean2, smile_data, 'Smile', no_smile_data, 'No smile')
```

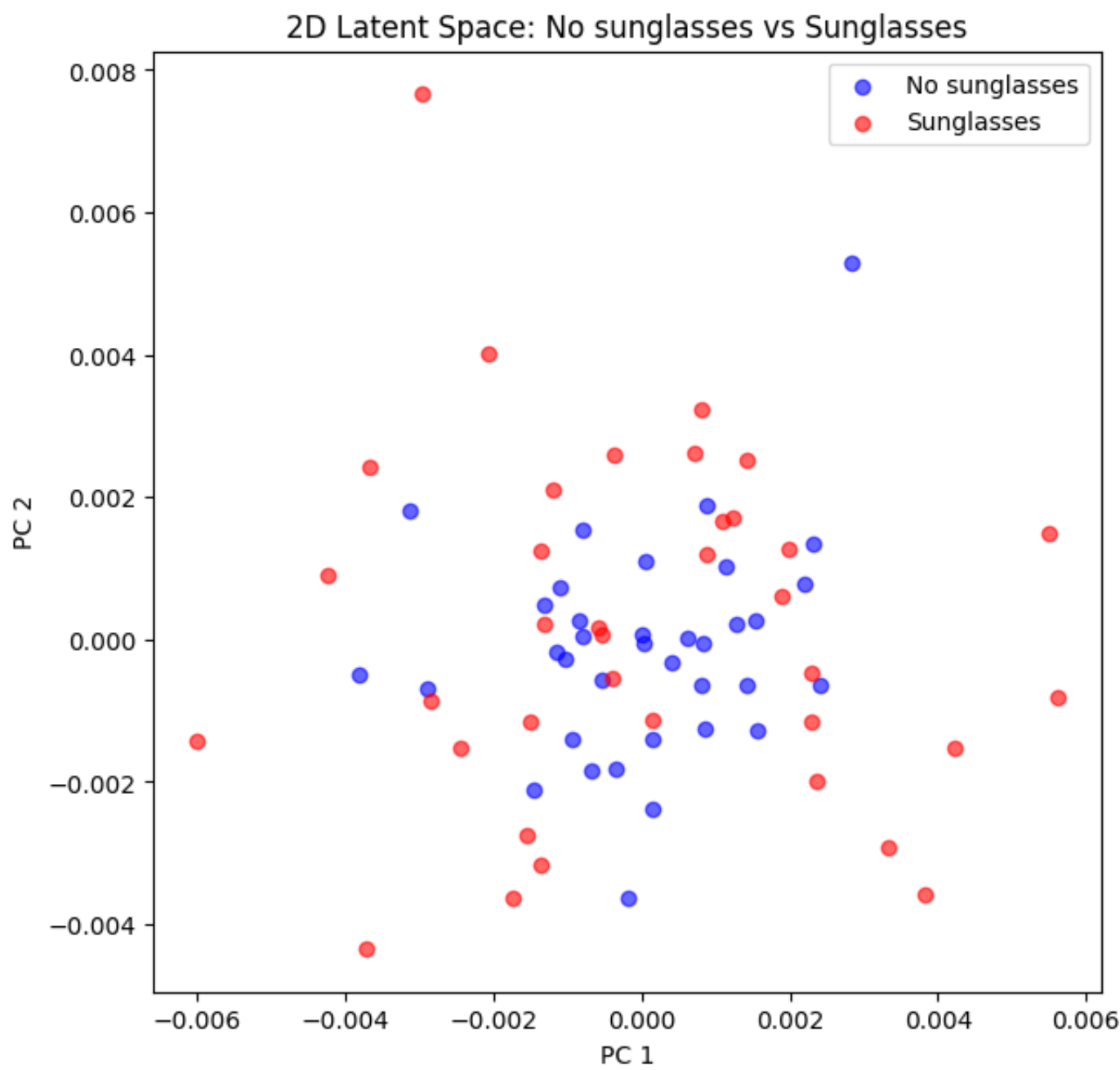
2/2 0s 180ms/step
2/2 0s 31ms/step



In [71]:

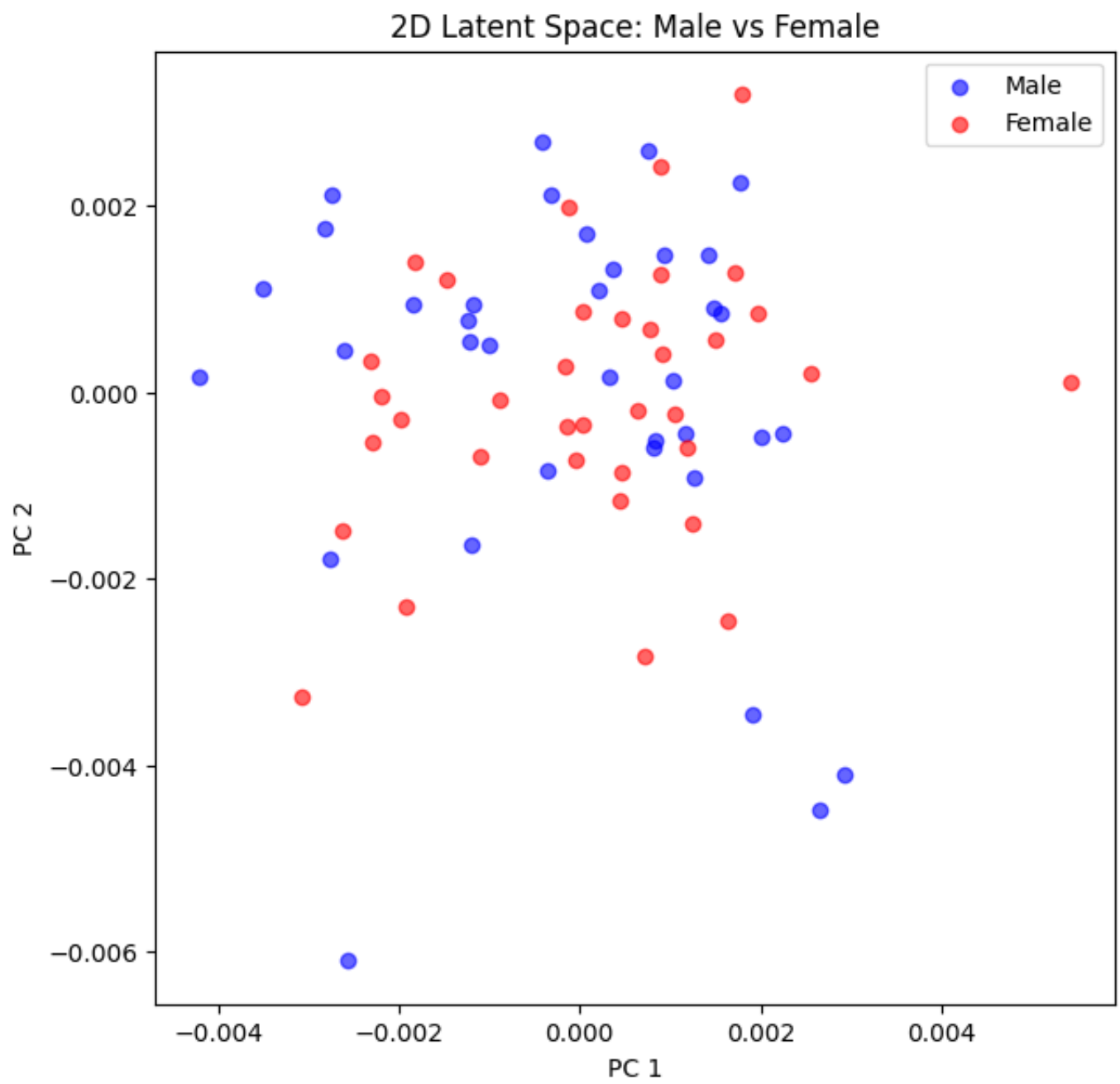
```
LatentSpace_2D(encoder_mean2, smile_data, 'No sunglasses', sunglasses_data, 'Sunglasses')
```

2/2 0s 21ms/step
2/2 0s 19ms/step



```
In [72]: LatentSpace_2D(encoder_mean2,male_data,'Male',female_data,'Female')
```

2/2 0s 14ms/step
2/2 0s 18ms/step



- Which model reconstructs the faces better?

- The model with a smaller factor on KLLoss (larger factor on Reconstruction Loss) reconstructs the faces better. Because it emphasizes on the reconstruction loss, it will prioritize minimizing the reconstruction loss. The reconstructed images are clearer and preserve more details from the original faces.
- The Model with a larger factor on KLLoss (smaller factor on Reconstruction Loss) may produce blurrier faces since they emphasize on latent space regularization. And theses faces almost look the same.
- Do the generated faces look different?
 - Yes. The model with a smaller factor on KLLoss (larger factor on Reconstruction Loss) generates faces that are clearer and closer to training images, but the latent space might be discontinuous, so many latent points may not correspond to realistic faces. But the model with a larger factor on KLLoss (smaller factor on Reconstruction Loss) produces faces that are blurrier and close to the average.
- How do the latent variable distributions differ?
 - The model with a smaller factor on KLLoss (larger factor on Reconstruction Loss), the data in the latent variable distributions are closer to each other and more centered around the origin, resembling a standard normal distribution.
 - The model with a larger factor on KLLoss (smaller factor on Reconstruction Loss) produces a more scattered latent space because the KL divergence in the loss function is less emphasized.
- Do the differences make sense? Can you explain what you observed?
 - Yes, the differences make sense to me.
 - A larger factor on KLLoss (smaller factor on Reconstruction Loss) increases the importance of the KL loss in the loss function, encouraging the latent codes to follow a Gaussian distribution and improving the generative capability of the model at the cost of reconstruction quality, while a smaller factor on KLLoss (larger factor on Reconstruction Loss) focuses more on accurate reconstruction.

Problem 2: Gaussian neighborhood regression (10 points)

In multivariate Gaussians, edges are about *direct* relationships after controlling for all other variables. The conditional law of a single coordinate X_i given the rest X_{-i} reveals which variables directly “enter” the model for X_i . Algebraically, this local behavior is encoded by the i th row/column of the precision matrix $\Omega = \Sigma^{-1}$. Graphical Lasso estimates a sparse precision matrix $\hat{\Omega}$ by maximizing the Gaussian log-likelihood with an ℓ_1 penalty on off-diagonal entries. Zeros in $\hat{\Omega}$ correspond to missing edges (conditional independences) in the learned graph.

Let $X = (X_1, \dots, X_p)^\top \sim \mathcal{N}_p(0, \Sigma)$, and write $\Omega = \Sigma^{-1}$. For $i \in \{1, \dots, p\}$, denote $-i = \{1, \dots, p\} \setminus \{i\}$.

2.1 Conditional law of one node.

Show that $X_i \mid X_{-i}$ is Gaussian with a linear mean in X_{-i} . Express the regression coefficients $\beta_{i \leftarrow -i} \in \mathbb{R}^{p-1}$ and the conditional variance $\sigma_{i|-i}^2$ in terms of Ω .

Hint. You may *complete the square* on $x^\top \Omega x$, or use a *Schur complement* derivation. After permuting coordinates so that i is first, partition

$$\Sigma = \begin{bmatrix} a & b^\top \\ b & C \end{bmatrix}, \quad a = \Sigma_{ii} \in \mathbb{R}, \quad b = \Sigma_{-i,i} \in \mathbb{R}^{p-1}, \quad C = \Sigma_{-i,-i} \in \mathbb{R}^{(p-1) \times (p-1)}.$$

Schur complement. If C is invertible (it is, since $\Sigma \succ 0$), the Schur complement of C in Σ is

$$S := a - b^\top C^{-1} b.$$

Block inversion formula. Then

$$\Sigma^{-1} = \begin{bmatrix} S^{-1} & -S^{-1} b^\top C^{-1} \\ -C^{-1} b S^{-1} & C^{-1} + C^{-1} b S^{-1} b^\top C^{-1} \end{bmatrix}.$$

2.1

Reorder the coordinates so that i is the first. and partition the covariance, $\Sigma = \begin{bmatrix} a & b^T \\ b & c \end{bmatrix}$, $a = \Sigma_{ii}$, $b = \Sigma_{-i,i}$, $c = \Sigma_{-i,-i}$.

From the result of conditional Gaussian, we know that $X_i | X_{-i} = x_{-i} \sim \mathcal{N}(\mu_i + b^T C^{-1}(x_{-i} - \mu_{-i}), a - b^T C^{-1} b)$

Since $X = (X_1, \dots, X_p)^T \sim \mathcal{N}_p(0, \Sigma)$, we know $\mu_i = \mu_{-i} = 0$. So $X_i | X_{-i} = x_{-i} \sim \mathcal{N}(b^T C^{-1} x_{-i}, a - b^T C^{-1} b)$

We can directly get $\beta_{i \leftarrow -i} = b^T C^{-1}$. $\sigma_{i|-i}^2 = a - b^T C^{-1} b$

From the block inversion formula: $\Sigma^{-1} = \begin{bmatrix} s^{-1} & -s^{-1} b^T c^{-1} \\ -c^{-1} b s^{-1} & c^{-1} + c^{-1} b s^{-1} b^T c^{-1} \end{bmatrix}$ $s = a - b^T c^{-1} b$

So, $\Sigma_{ii} = s^{-1} = \frac{1}{a - b^T c^{-1} b}$ $\Sigma_{i,-i} = -s^{-1} b^T c^{-1} = -\frac{b^T c^{-1}}{a - b^T c^{-1} b}$ we can get that $b^T c^{-1} = -\frac{\Sigma_{i,-i}}{\Sigma_{ii}}$.

Therefore, $\beta_{i \leftarrow -i} = b^T c^{-1} = -\frac{\Sigma_{i,-i}}{\Sigma_{ii}}$ $\sigma_{i|-i}^2 = a - b^T c^{-1} b = \frac{1}{\Sigma_{ii}}$

2.2 One neighbor at a time.

Fix $j \neq i$. First prove that the coefficient of X_j in the mean of $X_i | X_{-i}$ equals zero if and only if $\Omega_{ij} = 0$. Then deduce (Gaussian case) that

$$\Omega_{ij} = 0 \iff X_i \perp X_j | X_{-(i,j)}.$$

Hint: Use the result from 2.1. "iff" ($\iff$) means: prove **both** directions ($\Rightarrow$ and $\Leftarrow$).

2.2

From 2.1, we know that the coefficient of x_j in the mean of $X_i | X_{-i}$ is the j -th value in $\beta_{i \leftarrow -i}$, which is:

$$(\beta_{i \leftarrow -i})_j = \left(-\frac{\Sigma_{i,-i}}{\Sigma_{ii}}\right)_j = -\frac{\Sigma_{ij}}{\Sigma_{ii}}. \text{ So } (\beta_{i \leftarrow -i})_j = 0 \iff \Sigma_{ij} = 0$$

① if $\Sigma_{ij} = 0$, we know $(\beta_{i \leftarrow -i})_j = 0$, which means the conditional mean $E[X_i | X_{-i}]$ does not depend on X_j .

the conditional variance is $\frac{1}{\Sigma_{ii}}$, also not dependent on X_j .

So as a Gaussian distribution, the conditional distribution $X_i | X_{-i}$ does not depend on X_j , namely $X_i \perp X_j | X_{-(i,j)}$

② if $X_i \perp X_j | X_{-(i,j)}$, we know that $p(X_i | X_{-i}) = p(X_i | X_{-(i,j)})$. Since $X_i | X_{-i}$ and $X_i | X_{-(i,j)}$ are both Gaussian,

the means on the 2 sides must be equal for every x_{-i} . And we know $E(X_i | X_{-i} = x_{-i}) = \beta^T x_{-i} = \beta_j^T x_{-(i,j)} + \beta_j x_j$

Assume $E(X_i | X_{-(i,j)} = x_{-(i,j)}) = \beta_0^T x_{-(i,j)}$. We must have $\beta_0 = \beta_j$, $\beta_j = 0$ to make the equation to be true.

So $\beta_j = 0$, namely $(\beta_{i \leftarrow -i})_j = 0$. We can get $\Sigma_{ij} = 0$.

Therefore, $\Sigma_{ij} = 0 \iff (\beta_{i \leftarrow -i})_j = 0 \iff X_i \perp X_j | X_{-(i,j)}$

2.3 Factorization and graph structure

Suppose every coefficient in the mean of $X_i | X_{-i}$ is zero (i.e., $\beta_{i \leftarrow -i} = 0$). Show that the joint density factorizes as $p(x) = p(x_i) p(x_{-i})$. Briefly interpret the implication **for the graph structure**.

2.3

if every coefficient in the mean of $X_i | X_{-i}$ is 0, we know that $X_i | X_{-i} \sim \mathcal{N}(0, \sigma_{i|-i}^2)$, and does not depend on X_{-i}

Since the variance $\sigma_{i|-i}^2 = \frac{1}{\Sigma_{ii}}$ also doesn't depend on X_{-i} , we know X_i and X_{-i} are independent.

Therefore, the joint density of X_i and X_{-i} can factorize, namely $p(x) = p(x_i) p(x_{-i})$

Interpretation: Node i is disconnected with the rest of the graph.

Problem 3: Taking stock (15 points)

A joint distribution of data has a natural graph associated with it. When the distribution is multivariate normal, this graph is encoded in the pattern of zeros and non-zeros in the inverse of the covariance matrix, also known as the "precision matrix."

In class we demonstrated the graphical lasso for estimating the graph on ETF data. In this problem you will construct two different "portfolios" of stocks, and run the graphical lasso to estimate a graph, commenting on your results.

All of the code you might need for this is contained in the demo.

Data provided

As demonstrated in class, you will run on equities data downloaded from Yahoo Finance. Your job is to construct two "portfolios" of stocks, each with some organizing principle (e.g., sectors, regions, indices vs. equities). Each portfolio must contain **at least 20 stocks**. The page https://en.wikipedia.org/wiki/List_of_S%26P_500_companies has GICS sectors, which you may find useful for the glasso problem.

You are provided two CSV files containing stock prices **since 2021-09-10 (post-COVID start)**:

- [full_stock_daily.csv](#) (rows = dates, columns = **ticker** symbols; daily close)
- [full_stock_weekly.csv](#) (rows = dates, columns = **ticker** symbols; weekly close = last available trading day of the week)

Note on NAs. Some symbols begin trading after 2021-09-10, so their early entries will be **NA** . Handle missingness during preprocessing (see checklist below).

You can look up prices by ticker and date directly from these CSVs. If a needed symbol is **not** in the CSVs, use the provided **R file** `stock_price_from_yahoo_finance.r` to fetch it (via BatchGetSymbols) and merge it into your working data.

Preprocessing checklist (concise):

- Select your symbols; **align dates** across symbols.
- For the analysis window you choose, **filter** symbols with poor coverage (e.g., keep $\geq 90-95\%$ non-missing).
- Build a **complete-cases** matrix for Graphical Lasso (no NAs across chosen symbols/dates).

```
In [77]: import numpy as np
import pandas as pd
from sklearn.covariance import GraphicalLasso, GraphicalLassoCV
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import networkx as nx
from IPython.display import clear_output
from time import sleep
```

```
In [74]: daily = 'https://raw.githubusercontent.com/YData123/sds365-fa25/main/assignments/assn3/full_stock_daily.csv'
weekly = 'https://raw.githubusercontent.com/YData123/sds365-fa25/main/assignments/assn3/full_stock_weekly.csv'
daily_df = pd.read_csv(daily, parse_dates=True, index_col=0)
weekly_df = pd.read_csv(weekly, parse_dates=True, index_col=0)
```

```
In [91]: def get_data(tickers, is_weekly=True):
    df = weekly_df if is_weekly else daily_df
    df = df[tickers].dropna(axis=1, thresh=int(0.95*len(df)))
    df = df.dropna()
    dates = df.index

    logreturns = np.log1p(df.pct_change()).iloc[1:]
    clipped_logreturns = np.maximum(np.array(logreturns), -4*np.std(logreturns, axis=0))
    X = np.array(clipped_logreturns)
    X = (X - np.mean(X, axis=0)) / np.std(X, axis=0)
    return X, dates

def get_graphical_lasso(X, alpha=0.25):
    glasso = GraphicalLasso(alpha=alpha).fit(X)
    # glasso = GraphicalLassoCV().fit(X)
    # print(glasso.alpha_)
    Omegahat = np.around(glasso.precision_, decimals=5)
    return Omegahat

def plot_heatmap(Omegahat):
    for j in np.arange(Omegahat.shape[0]):
        Omegahat[j,j] = 0
    sns.heatmap(Omegahat, cmap='jet')
```



```

plt.show()
return

def plot_network(precdf, threshold=0.01):
    links = precdf.stack().reset_index()
    links.columns = ['var1', 'var2', 'value']
    links = links.loc[(abs(links['value']) > threshold) & (links['var1'] != links['var2'])]

    G = nx.from_pandas_edgelist(links, 'var1', 'var2', create_using=nx.Graph())
    pos = nx.spring_layout(G, k=1.7/np.sqrt(len(G.nodes())), iterations=20)
    plt.figure(figsize=(10,10))
    nx.draw(G, pos=pos, node_color='lightblue', node_size=600, edge_color='gray')
    nx.draw_networkx_labels(G, pos=pos, font_size=7)
    plt.title("Graphical Lasso Network")
    plt.show()
    return

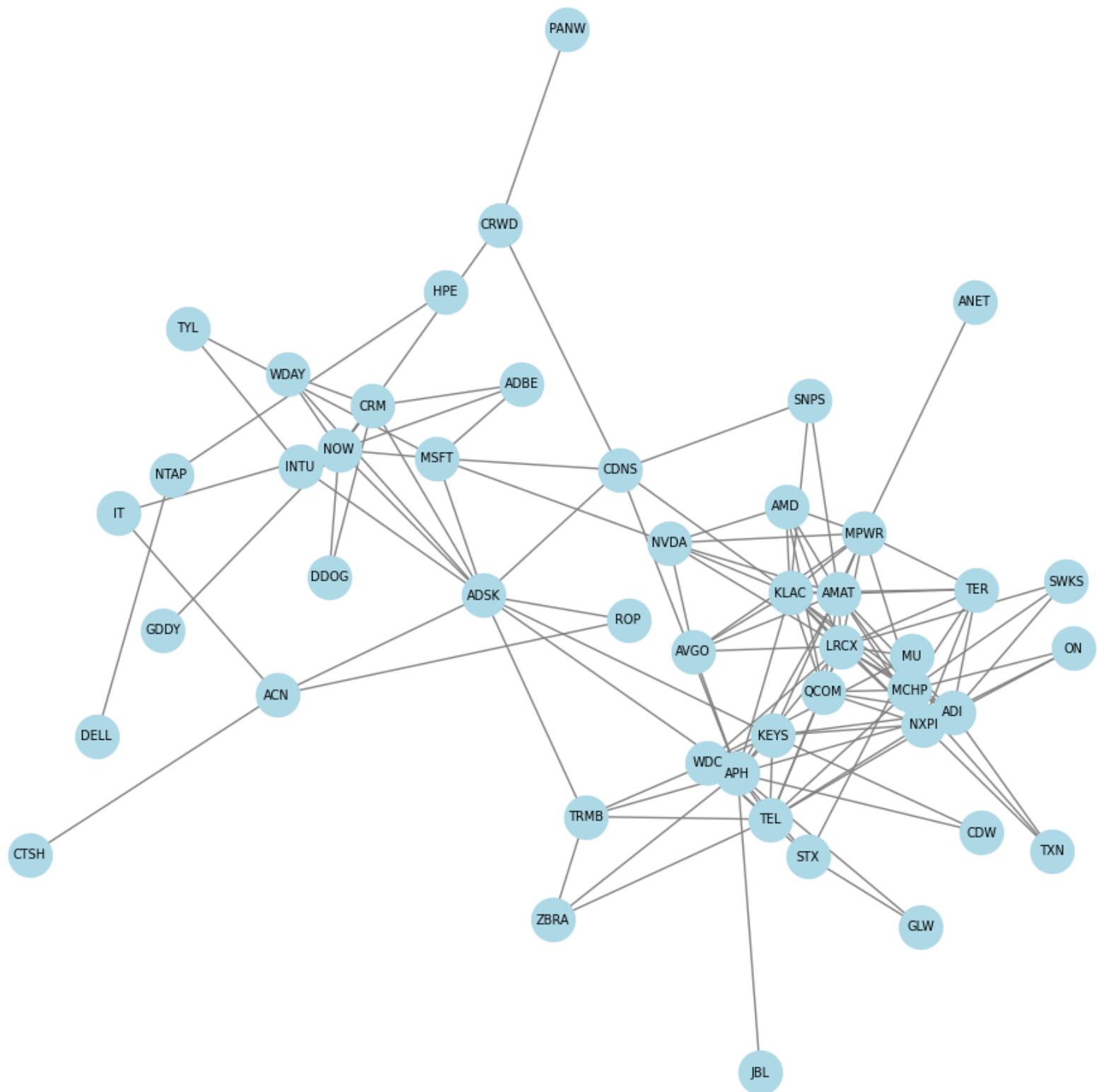
```

```

In [97]: tech_stocks = ['ACN', 'ADBE', 'AMD', 'AKAM', 'APH', 'ADI', 'AAPL', 'AMAT', 'APP',
                        'ANET', 'ADSK', 'AVGO', 'CDNS', 'CDW', 'CSCO', 'CTSH', 'GLW', 'CRWD',
                        'DDOG', 'DELL', 'EPAM', 'FFIV', 'FICO', 'FSLR', 'FTNT', 'IT', 'GEN',
                        'GDDY', 'HPE', 'HPQ', 'IBM', 'INTC', 'INTU', 'JBL', 'KEYS', 'KLAC',
                        'LRCX', 'MCHP', 'MU', 'MSFT', 'MPWR', 'MSI', 'NTAP', 'NVDA', 'NXPI',
                        'ON', 'ORCL', 'PLTR', 'PANW', 'PTC', 'QCOM', 'ROP', 'CRM', 'STX',
                        'NOW', 'SWKS', 'SMCI', 'SNPS', 'TEL', 'TDY', 'TER', 'TXN', 'TRMB',
                        'TYL', 'VRSN', 'WDC', 'WDAY', 'ZBRA']
X_tech, dates = get_data(tech_stocks, is_weekly=True)
Omegahat_tech = get_graphical_lasso(X_tech, alpha=0.6)
# plot_heatmap(Omegahat_tech)
precdf_tech = pd.DataFrame(Omegahat_tech, columns=tech_stocks, index=tech_stocks)
plot_network(precdf_tech, threshold=0.01)

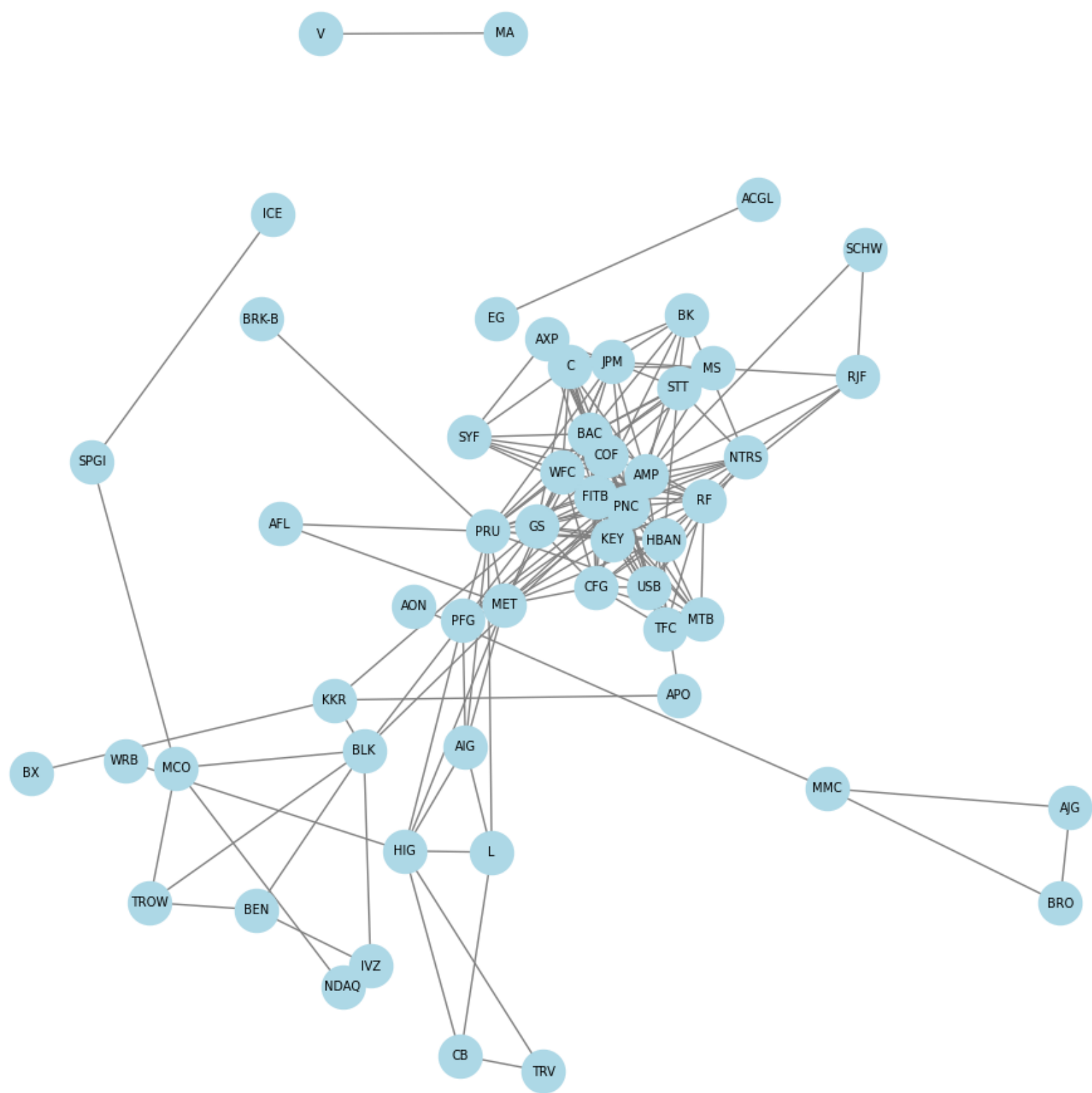
```

Graphical Lasso Network



```
In [103... # the symbol of BRK.B is 'BRK-B' in the dataset
fin = ['AFL', 'ALL', 'AXP', 'AIG', 'AMP', 'AON', 'APO', 'ACGL', 'AJG',
       'AIZ', 'BAC', 'BRK-B', 'BLK', 'BX', 'BK', 'BRO', 'COF', 'CBOE',
       'SCHW', 'CB', 'CINF', 'C', 'CFG', 'CME', 'COIN', 'CPAY', 'ERIE',
       'EG', 'FDS', 'FIS', 'FITB', 'FI', 'BEN', 'GPN', 'GL', 'GS', 'HIG',
       'HBAN', 'IBKR', 'ICE', 'IVZ', 'JKHY', 'JPM', 'KEY', 'KKR', 'L',
       'MTB', 'MMC', 'MA', 'MET', 'MCO', 'MS', 'MSCI', 'NDAQ', 'NTRS',
       'PYPL', 'PNC', 'PFG', 'PGR', 'PRU', 'RJF', 'RF', 'HOOD', 'SPGI',
       'STT', 'SYF', 'TROW', 'TRV', 'TFC', 'USB', 'V', 'WRB', 'WFC', 'WTW']
X_fin, dates = get_data(fin, is_weekly=True)
Omegahat_fin = get_graphical_lasso(X_fin, alpha=0.7)
# plot_heatmap(Omegahat_fin)
precdf_fin = pd.DataFrame(Omegahat_fin, columns=fin, index=fin)
plot_network(precdf_fin, threshold=0.01)
```

Graphical Lasso Network



Analyzing your portfolios

Your task is to analyze each portfolio using the graphical lasso, and comment on your findings. Here are the types of questions you should address:

- How did you choose the portfolio? How did you choose the date range and frequency (daily, weekly, etc.)? Remember, each of the portfolios must contain at least 20 stocks, and be organized in some reasonable way.
 - I choose the portfolio on Information Technology and Financials, including the data of all the stocks within the corresponding sector listed in the wikipedia page.
 - I choose the date range from 2021-09-10 (post-COVID start) to the latest time in the CSV.
 - I use the weekly frequency.
- Display the graph obtained with the graphical lasso, using networkx. How did you choose the regularization level? Does the structure of the graph make sense? Is it sensitive to the choice of regularization level? Is this the structure you expected to see when you designed the portfolio? Why or why not?
 - I choose the regularization level (λ) in the Graphic Lasso to be 0.5 after trying a list of possible values such as 0.75 and 0.25.
 - It is sensitive to the choice of regularization level.
 - If the λ is small, the graph will be dense with many edges, which makes it hard to interpret and may cause overfitting.
 - If the λ is large, the graph will be sparse, which may lose the genuine relationships between the stocks.
 - The structure makes sense and is what I expected to see.
 - The companies in Information Technology sector usually have higher internal connectivity because many of them

have strong correlations. Companies focusing on hardware and large platforms are more likely to be the central nodes with high degrees such as NVDA, MSFT.

- The companies in Financial sector have more subclusters. For example, companies in the sub-industry of Insurance Brokers like BRO, MMC, AON may be a subcluster, while companies in Transaction & Payment Processing Services like V and MA will be another subcluster.
- What are some of the conditional independence assumptions implied by the graph? Are some parts of the graph more densely connected than others? Why?
 - If $precision[i, j] \approx 0$, the node i and node j in the graph will have no edge connecting them, which means node i and node j are conditionally independent given all other stocks. And it might indicate that the correlation between the two stocks can be explained by other stocks, not by the direct interaction between them.
 - the IT sector has denser connectivity. This might be because that many IT companies will be influenced by common factors such as hardware, semiconductors, large platforms and so on, which leads to higher correlations and direct connections between companies.